上海大学

毕业论文（设计）

题 目：基于大模型的英语学习助手

专业(层次) 专升本

年 级 2023

班 级 02

学 号 234A0776

学生姓名 钱丁君

指导教师 刘福岩

上海大学继续教育学院毕业论文（设计）声明书

一、本人已经阅读并知晓毕业论文（设计）写作各项要求，包括毕业论文（设计）检测不合格者，毕业论文（设计）为不及格，由此造成的后果自负。

二、本人声明所呈交的毕业论文（设计）工作是由本人独立完成，在完成毕业论文（设计）时所利用的一切资料均已在参考文献中列出，不存在抄袭、剽窃、伪造、篡改、买卖、代写等学术不端行为，否则本人愿意承担一切责任。

三、本人已知晓《上海市本科毕业论文（设计）抽检实施细则（试行）》有关要求，对抽检不合格或存在学术不端行为的毕业论文，经查实后按规定承担相应责任。

四、本人完全了解上海大学继续教育学院有关保留、使用毕业论文（设计）的规定，即学校有权保留毕业论文（设计）及其复印件或电子件，允许毕业论文（设计）被查阅和借阅；学校可以公布毕业论文（设计）的全部或部分内容。

学 号：___234A0776_____

学生签名：_____

日 期：_____年____月____日

中文目录

摘 要 3

ABSTRACT 4

第一章 绪论 5

1.1 研究背景 5

1.2 研究意义 5

1.3 国内外研究与应用现状 5

1.4 研究内容与主要工作 7

1.5 论文组织结构 7

第二章 系统关键技术 9

2.1 Java 17 与 Spring Boot 9

2.2 Spring Security 与 JWT 10

2.3 MyBatis-Plus、MyBatis 与 MySQL 10

2.4 大语言模型与多模型兼容接口 11

2.5 缓存与异步任务执行机制 13

2.6 HTML5 音频与语音合成技术 14

2.7 模块化单体与架构治理 14

2.8 前端 ES Modules 与模块化组织 15

第三章 系统需求分析 16

3.1 可行性分析 16

3.2 用户角色与学习场景 16

3.3 功能需求 17

3.4 非功能需求 18

3.5 核心业务流程 18

第四章 系统总体设计 20

4.1 总体架构设计 20

4.2 功能模块设计 21

4.3 核心业务流程设计 21

4.4 数据库设计 22

4.5 接口与安全体系设计 24
4.6 数据一致性设计 25
第五章 系统详细设计与实现 27
5.1 用户认证与权限控制 27
5.2 多模型配置与模型调用 28
5.3 词汇大挑战与学习计划、公共词本 29
5.4 词汇学习卡与公共缓存 31
5.5 拼写容错与最佳匹配 32
5.6 单词本快照与笔记共享 33
5.7 文章学习模块 34
5.8 间隔复习与睡眠保护 35
5.9 前端模块化与交互设计 37
5.10 系统日志与异常处理 39
5.11 工程治理与可维护性 39
第六章 系统测试与结果分析 41
6.1 测试环境、数据来源与方法 41
6.2 功能测试与自动化验证 41
6.3 接口并发与首屏性能测试 42
6.4 拼写容错测试 43
6.5 本地使用情况与数据库分析 43
6.6 缓存与 AI 调用结果分析 44
6.7 技术指标达成情况与局限 44
第七章 总结与展望 46
7.1 工作总结 46
7.2 不足与展望 46
致 谢 48
参考文献 49
附　录 A 主要接口清单 51
附　录 B 词汇学习卡 JSON 字段说明 52

摘 要

英语词汇学习工具能够提供释义与复习提醒，但固定词库难以持续适应个人学习需求，通用大语言模型的生成内容也缺少与词表、笔记及复习记录衔接的机制。本文围绕这些问题设计并实现英语学习助手，将模型生成结果保存为可管理、可复习的学习材料。

系统采用前后端分离的模块化单体架构，后端基于 Java 17、Spring Boot 和 MySQL，前端使用 HTML、CSS 与原生 JavaScript 模块。管理员导入 Markdown 词表后，模型执行增量语义分析，学习计划据此选取主题相关词群并生成场景文章，每篇材料包含的核心学习词不超过 50 个。含义选择、跟敲和拼写检查形成逐词记录，并与单词本及间隔复习衔接。AI 网关按模型能力选择请求适配器和响应解析器，估算输入 Token 并预留输出空间；耗时生成由持久化异步任务执行，支持预约、取消和失败项重试。公共词卡缓存减少重复调用，个人快照保存笔记与学习状态。

本地验证中，100 并发只读接口均成功返回，浏览器业务数据就绪 P95 为 1989 毫秒，编辑距离为 2 的样本 Top-1 准确率为 97.17%。实践结果支持上述工程流程的可用性；由于学习数据主要来自单个用户，尚不能据此判断系统对长期词汇保持的普遍效果。

关键词：大语言模型；场景化词汇学习；间隔重复；模块化单体；个性化学习

ABSTRACT

Fixed vocabulary resources and general-purpose language-model conversations provide limited support for preserving generated materials alongside personal notes and review records. This thesis presents an English-learning assistant that stores model outputs as reusable learning resources and connects vocabulary lists, contextual practice, and spaced review.

The assistant uses a modular-monolith backend built with Java 17, Spring Boot, and MySQL, with a browser frontend based on HTML, CSS, and native JavaScript modules. Administrators import Markdown word lists, and incremental semantic analysis supports the selection of thematically related words for learning plans. Daily core vocabulary is distributed across contextual passages, each containing no more than 50 target vocabulary items. Meaning selection, typing, and spelling checks produce individual word-level records. An AI gateway selects request adapters and response parsers according to model capabilities, estimates input tokens, and reserves output capacity. Long-running generation is managed through persistent tasks with scheduling, cancellation, and failed-item retries. Shared vocabulary-card caching reduces repeated calls, while personal snapshots retain notes and learning progress.

Local verification covered read-only interfaces at 100 concurrent requests, browser loading, and spelling tolerance. All tested concurrent requests succeeded; the 95th-percentile time until business data became ready was 1,989 milliseconds, and top-1 matching accuracy for edit-distance-two samples was 97.17%. These results support the feasibility of the implemented workflow under local conditions. Learning records came mainly from one user, so the study does not establish a general improvement in long-term vocabulary retention.

Keywords: large language model; contextual vocabulary learning; spaced repetition; modular monolith; personalized learning

第一章 绪论

1.1 研究背景

移动互联网与云计算推动英语学习工具由电子词典和固定课程资源扩展至在线问答与个性化练习。大语言模型进一步提供了按需生成释义、例句和阅读材料的能力。潘教峰等从数据、知识与系统整合的角度讨论人工智能的应用价值，强调模型能力需要与领域数据及工程系统协同[1]。对词汇学习应用而言，生成内容还需与词条、笔记和复习记录建立关联，才能被持续使用。

预先整理的词库便于控制内容质量，但难以围绕学习者当前的词汇量、兴趣和目标即时生成语境。通用 AI 对话可以补充内容，其输出格式、个人进度管理和长期保存机制却未必符合学习流程。刘芹、谭佳豪指出，大语言模型在外语个性化学习中具有应用机会，同时存在信息真实性、技术依赖和学术规范等风险[2]。本课题因此将研究范围限定为英语词汇学习，重点处理生成内容的结构化、个人学习数据管理及复习调度之间的衔接。

1.2 研究意义

词汇学习需要同时处理释义、例句、搭配和语境，并通过练习检验能否主动回忆。分散练习的间隔与保持时间存在关联[11]，重复检索也有助于延迟回忆[12]。本文据此将生成材料与检索练习、学习记录及间隔复习连接起来，由模型提供内容，学习者完成判断、记录和复习。

系统实现涉及多模型接入、结构化输出解析、公共缓存与个人快照隔离，以及密钥存储等问题。这些问题也存在于编程学习和专业文献阅读等应用中。本文通过具体的英语学习流程考察上述机制的协作方式，其工程结果可为其他需要保存生成内容和个人状态的垂直应用提供参考。

1.3 国内外研究与应用现状

1.3.1 国外研究与应用

计算机辅助语言学习在国外的起步早于生成式 AI，早期注意力集中在电子词卡、语料库、在线反馈和移动学习这几条线上。Nakata 比较了多款词汇卡软件后认为，检

索、间隔、反馈以及学习项目管理四项，是决定软件效果的关键[13]。Schmitt 的综述则指出：词汇知识横跨形式、意义和使用三个维度，只给一条中文释义，不足以支持稳定掌握[14]。Webb 的实验结果与之呼应——接触次数增加确实能改善多维词汇知识，但各个维度所需的重复次数并不一致[15]。本系统的词汇卡字段设计和分阶段复习安排，主要依据的就是这几条结论。

生成式人工智能进入教育场景之后，研究的问题变了：从“如何推送固定资源”转向“如何动态生成内容、如何做交互式辅导”。Kasneci 等梳理了大语言模型在个性化解释、自动反馈和学习支持上的机会，也没有回避幻觉、偏见、隐私和过度依赖这几类风险[7]。Tlili 等以教育聊天机器人为案例，分析了生成式 AI 的机会与伦理挑战[8]。Lo 的快速综述给出的判断更谨慎：教学辅助、评价和学习支持都已有研究覆盖，但实证证据和长期影响还不足以支撑强结论[9]。Kohnke 等则讨论了 ChatGPT 在语言教与学中的适用边界[10]。综合上述研究可以认为：大模型适合产出多样化内容，而结构约束、数据治理和学习过程的组织，必须由应用系统自己承担。

1.3.2 国内研究与应用

国内已有多种成熟的词汇学习产品。百词斩、扇贝单词、不背单词、有道词典等应用在词卡、发音、例句、单词本和复习计划上都有成熟做法。模型服务的接入门槛也有所降低，DeepSeek、通义千问、Kimi、豆包等国产模型让中文学习应用能够接入生成式能力。朱永海等判断，生成式人工智能正把个性化学习从资源推荐推向学习内容生成、路径优化与多智能体协同[3]。国内研究同时开始检验大语言模型在外语知识查询、范例生成、纠错、测试和情境对话上的实际表现[2]。牟智佳等[4]从生成性目标导向出发，提出了大语言模型支持的个性化学习活动设计框架；王宇程[5]则对教育智能体赋能个性化学习的一般架构与特征做了系统剖析。

现有产品大体可以分成两类。一类沿用固定词库和既定复习流程，胜在稳定，生成能力有限；另一类直接做通用 AI 对话，内容丰富，却少了单词本、复习状态和长期数据沉淀。还有一些细节上的共性短板：只支持单一供应商、模型输出解析不稳定、API Key 保护不足，或者根本没区分共享生成内容与个人学习数据。本系统的工作重心就放在补齐这些工程边界上。

表 1-1 三类英语学习方式比较

比较维度	固定词库型工具	通用 AI 对话工具	本文系统
内容来源	预置词库	即时生成	缓存优先，必要时生成
结构稳定性	高	依赖自然语言输出	Prompt 约束 + JSON 解析校验
个人沉淀	单词本与状态	多为会话历史	个人快照、笔记与复习记录
复习机制	产品内置	通常没有	分阶段间隔与睡眠保护
模型接入	通常不可配置	单一平台	多供应商 OpenAI-compatible
安全边界	平台托管	平台托管	JWT、AES-GCM、脱敏与审计

1.4 研究内容与主要工作

需求分析覆盖学习者与管理员两类角色，由此划出个人信息、公共词表、词汇大挑战、语境精读、单词本、复习和系统管理七个产品模块。

公共词表以 Markdown 格式导入，导入后允许人工修正疑似断词，再进入发布与增量语义分析环节；场景候选因此按主题相关性组织，而不是照着词表顺序机械切分。

学习计划与周/月日历配套实现，场景摘要与详情采用分层加载。核心词每篇不超过 50 个，按均分策略分配；学习动作包括含义选择、跟敲和拼写检查三种。

AI 网关具备模型感知能力，请求适配、响应解析、上下文预算与输出限制均由供应商和模型的定义驱动；固定动作不携带长期会话历史。

公共词卡缓存与个人词条快照分开建立。个人笔记和复习进度因此不受公共缓存刷新的影响，减少重复调用成本；Markdown 笔记、语境精读和间隔复习同样在这一层完成。

安全与工程质量方面，完成了 Spring Security + JWT 鉴权、声明式权限、AES-GCM 密钥加密、可重试异步任务和 Outbox 日志，并对模块边界做了治理；后端编译、前端语法检查和文档质量验证一并通过。

1.5 论文组织结构

全文共七章。

第一章介绍研究背景、意义与国内外现状；

第二章说明项目采用的关键技术；

第三章开展可行性与需求分析；

第四章给出系统架构、模块划分、数据库、接口与安全设计；

第五章结合真实代码说明核心模块的实现；

第六章报告测试方法、结果与局限；

第七章总结工作并讨论后续研究方向。

第二章 系统关键技术

2.1 Java 17 与 Spring Boot

后端采用 Java 17 和 Spring Boot 3.3.6。Java 17 提供稳定的长期支持运行时、记录类和现代集合处理能力；Spring Boot 负责 Web 容器、参数校验、配置绑定、统一响应和依赖装配，使各业务域可以保持清晰的 API、应用、领域和基础设施边界。

耗时的词表分析、词卡、场景材料和阅读材料生成使用 Spring 事件、事务提交后监听与@Async 执行，并由有界 aiTaskExecutor 控制并发。数据库事务只保存任务和业务状态，不等待外部 HTTP 响应，从而避免大事务和请求线程长时间占用。

Java 17 提供文本块、记录类和 switch 表达式等语言能力。项目按业务需要使用记录类封装部分内部结果和事件，HTTP 请求与响应对象仍主要采用带有校验及文档注解的普通 Java 类。自定义 SQL 集中维护在 MyBatis XML 中，AI 调用场景使用枚举与 switch 表达式完成分支选择。

Spring Boot 通过自动配置和 Starter 装配 Web 容器、参数校验及配置绑定，Actuator 提供健康状态和指标入口。接口访问日志切面集中记录路径、用户、链路和耗时；权限切面在方法执行前校验声明式权限，以减少业务方法内重复的技术处理。

应用层显式定义事务边界。场景材料、词卡与词本关联分析先保存任务和状态，事务提交后再由事件与有界线程池触发模型调用。外部 HTTP 请求不持有数据库事务，避免模型等待时间扩大锁占用和请求线程开销。

表 2-1 系统技术栈

层次	技术	作用
JDK 版本	Java 17	后端语言与运行时
应用框架	Spring Boot 3.3.6	Web、配置、校验与应用装配
安全	Spring Security + JWT	无状态认证与接口访问控制
数据访问	MyBatis-Plus 3.5.12	Mapper、条件构造器、审计字段与乐观锁
数据库	MySQL	用户、学习资产、AI 缓存和日志持久化
模型接口	OpenAI-compatible Chat Completions	统一接入不同供应商

层次	技术	作用
前端	HTML/CSS/ES Modules	轻量模块化单页交互
文档接口	SpringDoc	接口浏览与联调辅助

2.2 Spring Security 与 JWT

登录采用 Spring Security 与 JWT。JwtAuthenticationFilter 登录后校验 Bearer 令牌的签名、issuer 和过期时间，校验通过后把用户身份同时写进 SecurityContext 和 CurrentUserContext。这样做的好处是业务接口不用再层层透传 Authorization 参数，应用服务在当前线程里直接就能拿到登录用户。

管理员接口只需要挂一个 @RequirePermission 注解声明 SYSTEM_ADMIN 权限，真正校验由权限切面统一做，方法里不用写判断。JWT 的声明按 RFC 7519 组织[16]。模型 API Key 用 AES-GCM 加密后入库，列表接口只返回脱敏值——以限制凭据在前端状态和业务日志中的暴露。

身份传播上定了一条规矩：令牌只解析一次。JWT 过滤器做完签名、签发方和过期校验，就把用户身份放进安全上下文和当前用户上下文，后面应用服务和审计基础设施都在线程内直接取。既不用重复传授权参数，也不用反复查用户表，凭据泄露面和重复查询开销一起降下来了。

供应商 API Key 在后端用 AES-GCM 加密存储，列表接口只给脱敏值。连接测试采用最小请求——只发一个最小请求去探供应商，不走完整学习流程，以控制验证配置的调用成本。生产与预发布环境启动时，如果检测到还在用开发默认的签名密钥或加密密钥，直接拒绝启动，避免默认凭据被用于正式环境。对外则统一走异常与稳定错误码，接口返回的是可读的业务语义，而不是技术堆栈。

2.3 MyBatis-Plus、MyBatis 与 MySQL

数据层是 MySQL 8 + MyBatis-Plus + MyBatis XML 的组合，职责划分如下：单表增删改查交给 MyBatis-Plus 的包装器，逻辑删除和乐观锁也用它；多表摘要、分页列表、批量写入这些则用 XML 里的显式 SQL，以便明确控制查询和批量处理；服务循环里逐条查或逐条改的写法受到工程规则约束。主键用 Snowflake，后端是 Long，序列化到 JSON 时统一转成字符串，否则 JavaScript 那边超过 2^53 就丢精度。

库里放的东西有用户、公共词表、个人词本、学习计划、场景材料、逐词进度、异步任务和 AI 审计记录。业务不变量尽量通过数据库约束维护：同一计划内的单元序号、同一单词本里的归一化词条，都由唯一约束保证。组合索引则是照着高频查询路径建的——日历摘要、待复习队列、任务状态分页。

结构演进交给 Flyway。空库跑基线脚本直接建出完整初始结构；如果是已经有数据的存量库，就按既定版本打基线，之后的迁移按版本号顺序往后追加。已经执行过的迁移不会被重写——以维护多环境结构的一致性及迁移的可追溯性。完整结构、种子数据、历史升级脚本三者分开存放，执行顺序由目录里的说明文件约束。

性能上只有一条硬原则：不在循环里执行 SQL。学习计划详情是一次批量加载，把单元、材料、词条、进度和检测记录一起装配出来，不做逐单元的 N+1。场景完成度同理，先把通过记录批量读出来，再在内存里分组。更新走批量 SQL，大批次在服务层分块。分页列表的响应体刻意保持精简，日历摘要、待复习队列和任务状态分页等高频路径配置组合索引。

2.4 大语言模型与多模型兼容接口

场景文章和词卡解释主要由模型生成，词表与个人笔记分别来自导入资源及用户输入。以 ChatGPT 为代表的这类模型具有文本理解、内容生成与问答能力，国内的 DeepSeek、通义千问、Kimi、豆包也提供可接入的模型服务，垂直学习场景的提供了内容生成能力。但通用聊天工具存在三方面限制：输出格式不统一、生成内容缺少长期管理机制、没法结合个人进度安排复习。本系统要做的，就是将生成能力纳入结构化数据和可追踪任务里，使生成内容得到保存。

接入层统一用 OpenAI 兼容的对话补全协议。消息角色、温度、结构化输出这些参数都是通用的，不同供应商因此能走同一套接口，前端可以直接切模型，业务代码里不会出现“如果是 A 供应商就怎样”的分支。目前 DeepSeek 和 Kimi 各自配了请求适配器和响应解析链。模型管理这边记录调用次数、成功失败数、Token、平均延迟和最近调用时间；普通用户只能看到启用模型的最小信息，密钥一类的一概不给。

AI 网关把供应商、模型、协议都当成能力元数据来处理，请求构造和响应处理严格分家。请求阶段，请求适配器按供应商和协议拼请求体。响应阶段，响应解析器先解析供应商的响应封装，再由场景编解码注册表按调用场景解包、把字段别名归一化、校验必需根字段，业务服务拿到的已经是规范化结构。每次调用必须声明场景枚

举，枚举里写清楚三件事——响应是否必须是 JSON、哪些根字段必须存在、固定动作允许带进提示词的请求变量。这些约束用于不让无关对象和历史对话混进模型输入。

上下文控制使用模型定义中的实际窗口和输出上限，并以字符规则保守估算输入 Token，而非通过供应商分词器精确计数。系统在安全窗口内为输出预留空间，输入过大时拒绝请求或由上层拆分批次。固定生成动作仅携带当前任务所需数据，需要连续对话时才复用会话。模型 HTTP 日志不记录完整响应正文，审计内容按配置脱敏和截断。

提示词没有写死在代码里，而是存库加变量渲染。模板按系统、用户、分析三类组织，每条都带变量定义、示例输入和示例输出；{{变量名}} 这种占位符由渲染器在运行时替换成字符串或者 JSON 对象值。这么安排的收益很直接：改提示词不用改代码、不用重新部署，管理员在运行期通过接口就能更新模板并验证示例输出。牟智佳等[4]指出，大语言模型支持的个性化学习活动设计需要灵活的提示词管理和生成性目标导向机制；本系统的模板化管理至少提供了提示词配置机制——不同学习场景的模型调用因此有了可追溯的提示词版本。

调用场景一共八种，写在一个枚举里：通用对话、模型连接验证、单条与批量词卡生成、词本关联分析、语境精读材料生成、场景单元生成、场景相关词生成。每种场景声明三类约束——响应是否必须为 JSON、必须存在的根字段（比如词卡生成要 term 和 definitions）、固定动作允许带进提示词的请求变量。枚举上还有一个独立动作判定方法，用来把固定生成任务和对话任务分开：词卡生成、关联分析、材料生成这类固定动作只带当前任务的数据，不会把同一学习场景之前的对话历史再发给模型一遍；只有通用对话，在当前场景确实需要连续上下文时才复用会话消息。王宇程[5]提到教育智能体需要按任务类型选择交互策略，本系统的场景枚举和独立动作判定将该思路落实为调用场景的代码约束。

模型响应可能包含额外格式，所以响应处理分两步走。第一步由供应商解析器把 JSON 从文本里恢复出来——有的模型会带 Markdown 代码块，有的前后还包含额外说明。第二步交给场景编解码注册表：先解包根包装对象（scene、data、result、material、card 这些都可能），再按别名字段归一化（word 归到 term，meanings 归到 definitions，example_sentences 归到 examples），最后校验该场景的必需字段。这里需要说明：如果模型返回的是语法合法但缺必需字段的 JSON，注册表抛的是“结构不完整”错误，而不是直接判任务失败。原因是这类缺字段属于上游的非确定性输出，可尝

试补充格式约束后重新请求，调用方因此可以定点重试。吴永和等[6]从多智能体协作与自适应角度讨论了大模型教育应用的技术路径，本系统这套归一化和契约校验，在多模型协作下提供结构化输出校验。

审计和重试是两件事，但都为了同一个目标：出问题时能查，且不会因为查问题而影响业务结果。审计处理中，提示词和响应正文都要截断加脱敏——内容默认截到十二万字符，错误信息截到一千字符；API Key、密码、JWT、完整提示词、完整模型响应一律不进日志。自动格式重试的范围较窄：只有结构化响应缺字段这一类失败才重试，做法是在保持业务输入不变的前提下给模型追加一句纠正要求，默认最多两次；网络问题、配额问题、内容安全问题都不对同一输入重试。还有一层隔离——审计落库写失败，不改变已成功的模型结果。诊断信息缺失是一回事，接口业务语义被改是另一回事。

2.5 缓存与异步任务执行机制

缓存用于减少重复模型调用。公共词卡按归一化词条存，普通查询优先走缓存；场景生成时先把不需要词卡的词和已命中的词过滤掉，剩下的缺失词按批次交给模型，失败项可以重试。词本关联分析的思路一样——标准化的单词先复用历史有效结果，只对缺失词调模型，出来的部分结果立刻落库，重试时只处理没完成的那些。

公共缓存和个人数据必须分开，二者具有不同的数据归属和更新规则。公共词卡是共享的，用户把一个词加进单词本时，系统会存一份个人词卡快照；以后公共缓存被重新生成，也不覆盖已保存的个人内容、笔记和复习进度。单词本词条单独保留个人学习状态，相关词按同义、反义、词族组织，搭配另外存放，共享数据和个人数据因此不会互相污染。

耗时任务全部走异步，用有界线程池加可重试任务模型。AI 线程池的大小、队列、存活时间、停机等待时间都能配。拒绝策略特意做了限制：不会把昂贵的模型任务退回请求线程执行——否则模型等待仍会占用请求线程。可重试的工作具备几个特征：原子领取、条目状态、幂等写入、终态判定，失败项可以单独重试。任务取消时，已经完成的结果保留，不回滚。

业务日志先写入 Outbox 并发布事件，事务提交后由异步监听器批量、幂等地写入正式日志表，恢复调度器重试尚未消费的记录。日志记录或持久化失败不改变已经成功的业务结果。AI 任务与审计日志分别使用 aiTaskExecutor 和 auditLogExecutor，二者

采用一致的 MDC 装饰方式传播请求及链路上下文，避免日志处理与模型生成争用同一工作队列。

2.6 HTML5 音频与语音合成技术

发音功能由前后端共同完成：前端用 HTML5 Audio，后端提供音频资源服务。单词发音统一走 MP3，前端用 HTMLAudioElement 做播放、预加载和音频对象缓存，按英式、美式口音挑资源。后端按词条加口音定位本地文件，没命中就去下载远程音源，落盘时先写临时文件再原子移动，避免下到一半的半成品被读到。音频接口设了长期 HTTP 强缓存；后端资源服务不可用时，前端还能降级去访问公共发音地址，维持基础发音能力。

句子和短文本要即时朗读，就没必要等服务端生成文件了。这类功能调用浏览器的 Web Speech API，用 SpeechSynthesis 和 SpeechSynthesisUtterance 找本地可用的英语语音，按英美口音筛音色，语速和音调都可以设。朗读前会把 Markdown 标记和学习注解先清掉，再按句子切分，由浏览器执行朗读。

较长的场景文章改用阿里云 NLS 语音合成服务。后端通过 WebSocket 提交清洗后的文本，按句子边界和最大字符数分片合成 16 kHz MP3，并用异步任务记录生成状态；任务完成后把音频持久化到本地，配合防重复锁和幂等写入，避免同一材料被反复合成。播放端用 HTMLAudioElement 做暂停、继续和倍速控制；答题反馈音效不走文件，直接由 Web Audio API 的 AudioContext、OscillatorNode 和 GainNode 即时合成，音效出异常也不影响主学习流程。

2.7 模块化单体与架构治理

架构上是按业务域切分的模块化单体——没有拆成微服务，进程只有一个，但边界按域划清楚。后端分身份、词汇、学习、阅读、任务、系统六个业务域；AI 那部分再按 Agent、模型、会话、提示词分；供应商协议、请求适配、响应解析全部收在网关层。每个业务域内部是四层：接口层（控制器、请求、响应）、应用层（用例服务、事务边界、编排）、领域层（实体、聚合、枚举、常量）、基础设施层（Mapper 接口）。

分层职责与依赖方向通过架构测试持续校验：接口层依赖应用层，应用层依赖领域层，基础设施层也依赖领域层；控制器不能直接访问 Mapper，领域代码不许反向依

赖接口层或基础设施层，每个应用类只访问本域的基础设施，跨域一律走窄化过的应用契约。该规则还限制了“横向目录”的回潮——controller、service、mapper 这类根级包不许再建，实体只能待在领域实体包，枚举只能待在领域枚举包，Mapper 只能待在基础设施的 Mapper 包。

依赖方向之外还有两类检查：一个源文件超过既定行数，必须按渲染、状态、策略或持久化职责拆开才允许合并；对外接口和关键领域概念必须有中文说明，文档缺失会导致相关验证不通过。

2.8 前端 ES Modules 与模块化组织

前端采用 HTML、CSS 与原生 ES Modules，开发时可直接运行源文件；发布构建执行模块检查并将静态资源复制至 dist 目录，不涉及框架编译或代码打包。功能按业务域组织，复杂场景进一步拆分纯业务模型、预览数据与 API 适配模块，入口文件负责装配。

共享层放的是各域都要用的东西：API 请求封装、弹窗、标识符字符串化、Markdown 渲染、本地偏好存储。标识这块前后端有约定——后端把长整型统一序列化成字符串，前端把所有标识当成不透明字符串处理，两边一起避开 JavaScript 的精度问题。学习页和复习页共用同一份个人 Markdown 笔记，所以两个场景下的内容是实时同步的。

为支持无数据库环境下的流程演示，所以前端留了一个预览模式，本地未连接数据库时也可演示主要产品流程。性能上做了分层加载：计划首页和日历先拿轻量摘要，点“开始学习”或“回顾”时才按计划与单元标识去取完整文章、核心词、词卡和学习状态，首屏不用一次性拉大字段。

第三章 系统需求分析

3.1 可行性分析

技术实现主要依托 Spring Boot、MyBatis-Plus、MySQL 和浏览器标准 API，模型通过兼容 Chat Completions 的接口接入。运行成本主要来自模型调用，因此系统复用词卡与文章缓存，并只分析缺失词条。页面按学习者常用的单词本、场景学习和复习任务组织，管理员通过独立入口维护公共资源。仍需处理的风险包括结构化输出不完整、供应商网络波动和密钥泄露，分别由契约校验、任务重试与缓存、加密存储和权限控制降低影响；生成内容的正确性与学习依赖问题仍需要人工复核及后续研究。

3.2 用户角色与学习场景

角色只有两种：普通用户和系统管理员。没登录的人只能注册和登录。普通用户可以维护个人资料、个人单词本、学习计划和学习设置，做词汇大挑战、语境精读和复习。系统管理员通过系统管理维护用户、公共词本、AI 异步任务、模型、Agent 与模板、系统日志和 AI 会话。公共词本的导入、修改、发布、删除只有管理员能做，普通用户只能看已发布的内容，拿它去建自己的学习计划。AI 调用场景至少包括词卡生成、公共词本关联分析、场景材料、阅读材料和批量词卡。

图 3-1 展示系统参与者及其用例。学习者维护个人词本、计划和复习记录，管理员维护公共词表、模型配置及异步任务。外部模型接收当前生成动作所需的词汇与目标信息，不直接访问学习数据库。

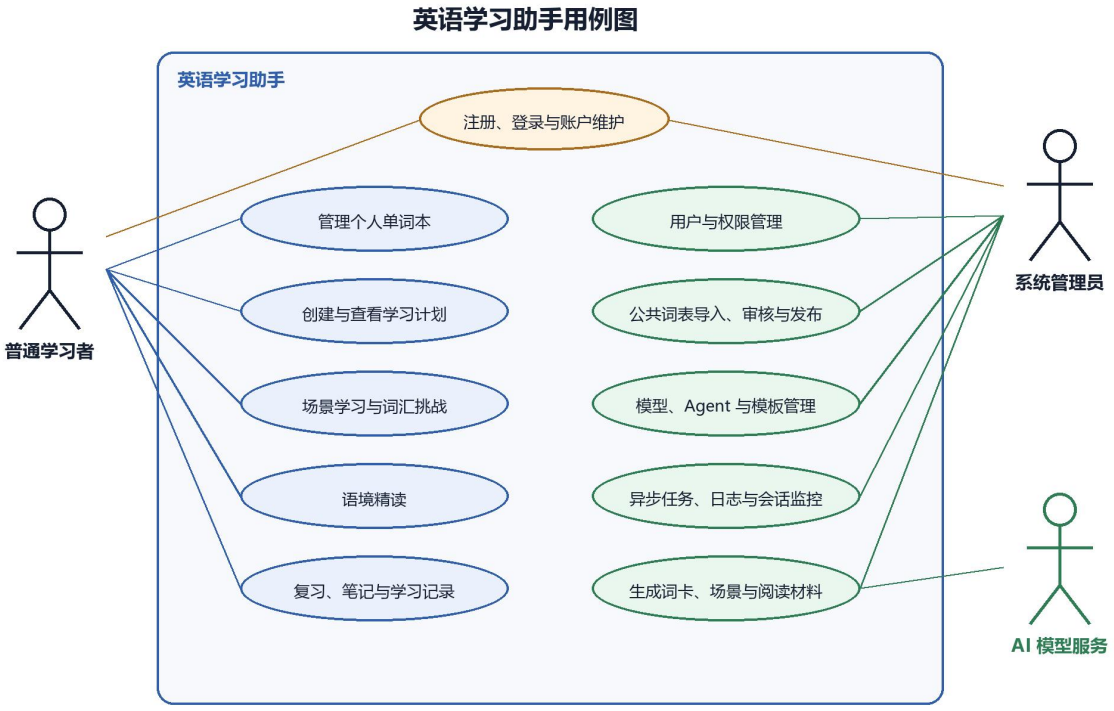


图 3-1 英语学习助手用例图

3.3 功能需求

表 3-1 主要功能需求

编号	功能域	主要需求
F01	认证与账户	注册、登录、退出、资料维护、学习设置及用户上下文
F02	公共词表	Markdown 导入、数据源类型、疑似断词修正、发布与增量关联分析
F03	学习计划	学习目标、公共词表、计划状态、周/月日历和场景材料生成
F04	词汇大挑战	相关词群文章、待挑战词汇、含义/跟敲/拼写检查和逐词记录
F05	词卡与缓存	结构化词卡、批量生成、拼写容错、公共缓存与个人快照
F06	个人单词本	多词本、词条增删改、状态筛选、移动/复制和 Markdown 笔记
F07	语境精读	选词、难度与字数、文章复用、三阶段阅读检测和专注模式
F08	复习	到期队列、重新组题、跟敲、自评、复习排期和历史记录
F09	AI 管理	模型、Agent、模板、会话审计、连接测试和异步任务监控

编号	功能域	主要需求
F10	个人偏好	学习 Agent、提示词模板、语音参数和跨学习入口偏好复用
F11	系统治理	用户中心、声明式权限、系统日志、异常处理与任务人工处置

3.4 非功能需求

表 3-2 非功能需求与验证方式

类别	目标或约束	验证方式
安全性	业务接口默认认证；管理员能力声明权限；API Key 不得明文回显	安全配置、权限切面与 DTO 走查
一致性	公共缓存刷新不得覆盖个人笔记、掌握度、排期和场景历史	快照、版本与数据归属逻辑检查
可用性	错误信息可理解；破坏性操作二次确认；学习入口支持继续上次进度	业务流程与界面走查
兼容性	不同供应商和模型使用独立适配、解析及上下文配置	模型定义与网关单元测试
可维护性	按业务域模块化；统一异常、枚举、中文说明和架构规则	ArchUnit 与文档治理测试
性能	列表加载摘要小对象，详情按需加载；批量查询避免 N+1	接口设计、SQL 与 Mapper 检查
成本	固定 AI 动作精简上下文；共享缓存；批量任务可预约低价时段	Token 审计与任务记录
可观测性	接口耗时、业务日志、AI 调用和任务状态可追踪	切面日志、系统日志与任务中心

任务书中的缓存命中率、响应时间和并发数属于部署环境指标，受词汇分布、模型供应商、网络、数据库规模和硬件影响。

3.5 核心业务流程

词汇学习的主流程是：管理员导入并审核公共词表，普通用户选择词表创建学习计划，系统依据学习目标规划场景和掌握要求；学习者按周/月日历打开场景材料，阅读文章和相关词汇，完成词汇检查，保存逐词记录并进入后续复习。日历和场景列表先加载摘要数据，点击学习或回顾时再按需加载完整材料。

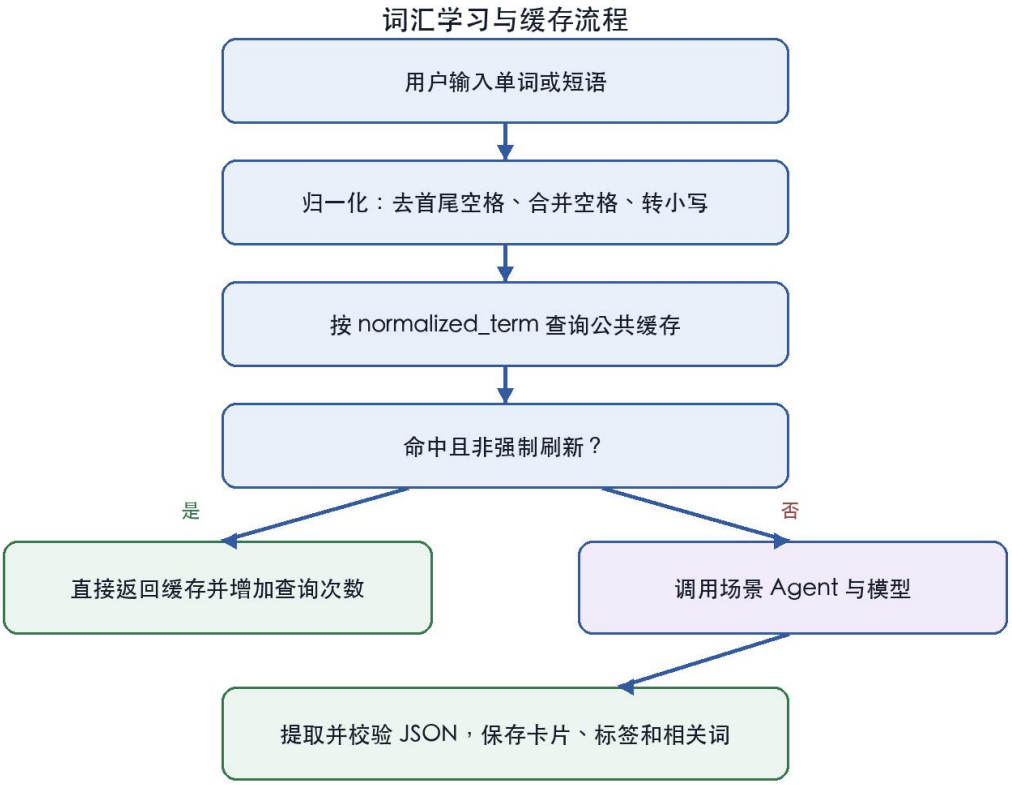


图 3-2 词汇学习与缓存流程

语境精读先校验所选词条属于当前用户和当前单词本，再抽取核心词义，以用户、词本、词汇组合、字数、难度和备注计算缓存哈希。缓存未命中时调用 english_article 场景 Agent，保存文章、译文、词汇用法、语法点、练习与建议。复习流程则先查询到期词条；若用户主动重启复习，只生成本轮队列而不改变正式排期，只有提交结果后才写复习记录并更新词条状态。

第四章 系统总体设计

4.1 总体架构设计

总体架构分成七块：浏览器表示层、接口与安全层、业务域应用层、领域层、基础设施层、数据存储，以及外部模型服务。浏览器通过 HTTP 与后端通信，除登录注册外由安全规则决定是否需要 Bearer JWT。请求进来后，安全过滤器解析令牌并建立 CurrentUserContext；Controller 只做接收参数、声明权限、组织响应；Application Service 编排用例；Domain 保存业务规则；Infrastructure 通过 MyBatis-Plus 和 XML Mapper 访问 MySQL。外部模型仅接收当前动作所需的数据，密钥和 Authorization 不进业务审计内容。

模块边界的存在，是为了把不稳定的外部生成能力纳入可审计的业务流程。数据库事务只负责保存任务、状态和业务数据，不在事务内等待模型的 HTTP 响应；提交后事件把外部工作交给有界的 aiTaskExecutor。这样一来，即使模型暂时不可用，已缓存的词卡、个人快照、场景历史、文章记录和复习进度仍可读取，失败的任务也能从未完成的步骤接着跑。

浏览器前端经 API 与安全层进来，请求落到按业务域切分的应用服务上，规则与持久化分别交给领域对象和基础设施，模型供应商之间的差异由 AI 网关统一吸收，学习资产、任务和审计数据都留在 MySQL 里可追溯，总体架构如图 4-1 所示。

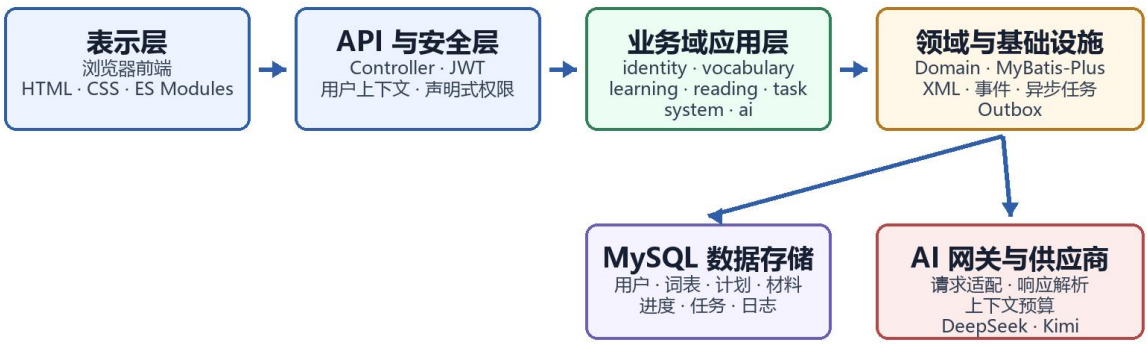


图 4-1 系统总体架构

4.2 功能模块设计

表 4-1 功能模块划分

模块	后端职责	前端职责
身份与权限	用户、JWT、用户上下文和声明式权限	登录注册、账户资料、学习设置和管理员入口
AI 能力	模型/供应商、Agent、Prompt、会话和模型网关	模型管理、Agent/模板、会话诊断和连接测试
词汇与词本	公共词表、关联分析、词卡缓存、个人快照与笔记	公共词表查看、个人单词本、词卡详情和导入审核
词汇大挑战	学习计划、场景编排、材料、挑战评测和学习记录	周/月日历、场景摘要、沉浸学习和词汇挑战
语境精读	阅读材料、缓存、进度、练习评分和学习曝光	通读、词汇精讲、阅读检测和专注模式
异步任务	任务、步骤、尝试、原子领取、预约、取消与重试	任务状态、进度、失败原因和人工处置
系统支撑	Outbox 日志、接口监控、异常映射和数据审计	用户中心、系统日志和运营查询

4.3 核心业务流程设计

核心流程从公共词表准备开始。管理员导入 Markdown 词表，经格式校验、疑似断词人工审核和发布后，系统以异步任务执行增量语义分析。学习者选择已发布词表并填写学习目的，系统依据标签和相关性组织候选词，按日期计算目标量，并把每日核心词均分至每篇最多包含 50 个核心学习词的场景材料。

生成任务成功后，学习者在同一单元中完成文章阅读、核心词挑战和 Markdown 笔记；每次答题写入逐词学习记录并更新掌握状态，待复习词进入间隔复习队列。任务失败时保留任务、步骤、尝试次数和错误原因，可由发起者或管理员在任务中心继续执行，业务流程如图 4-2 所示。

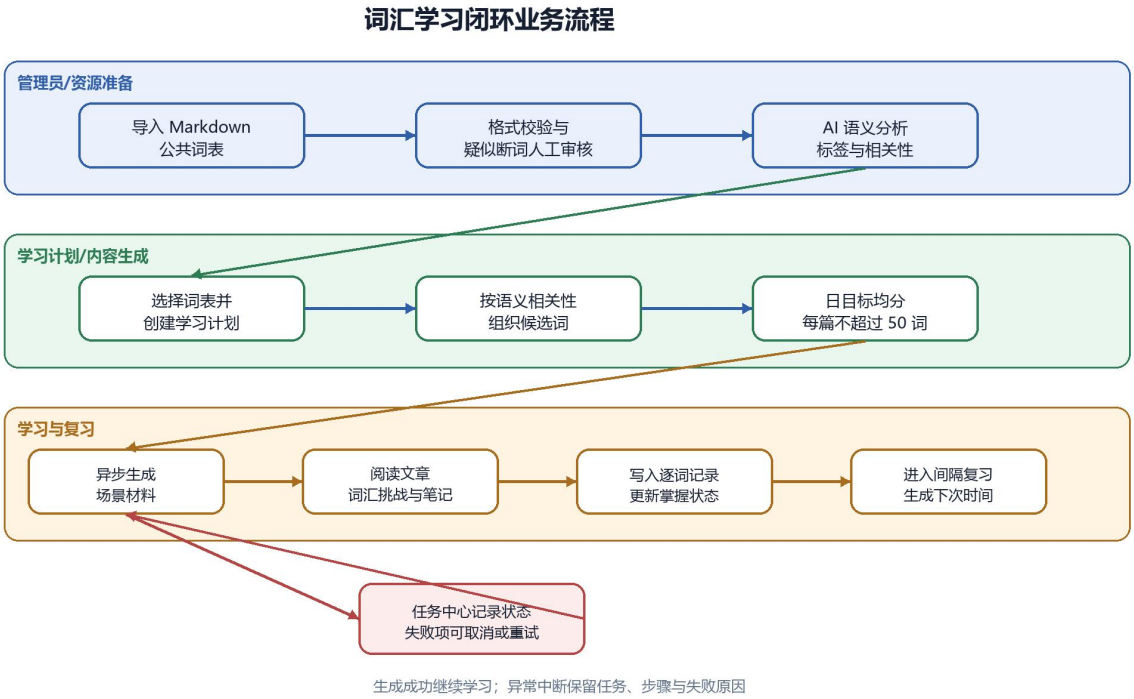
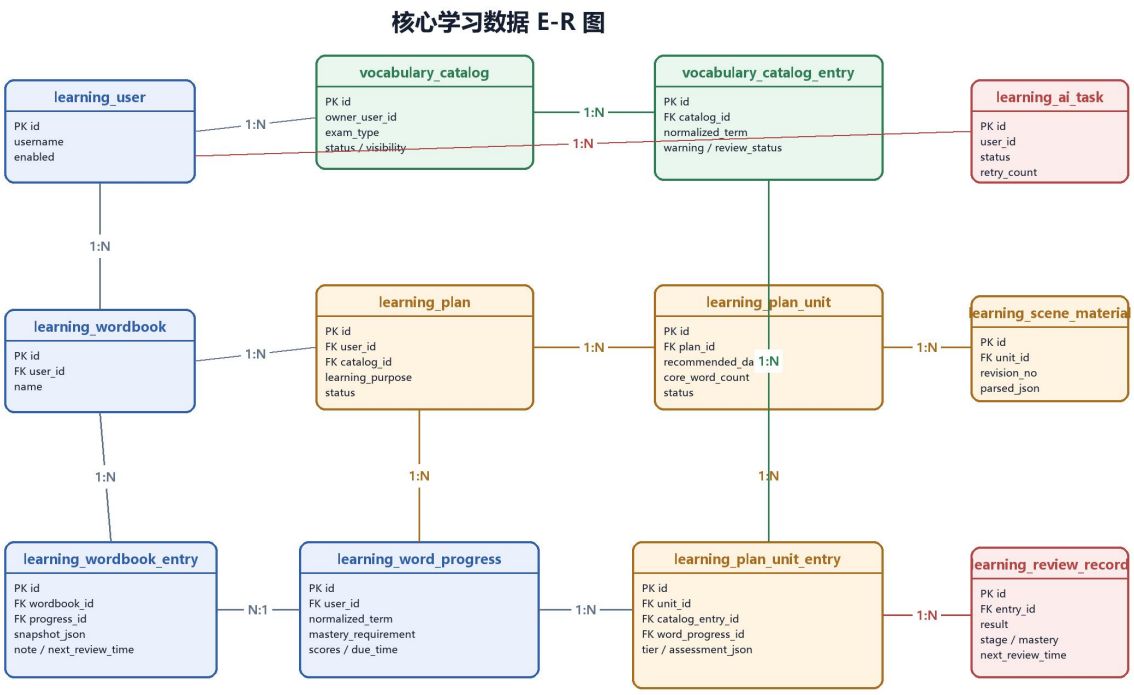


图 4-2 词汇学习闭环业务流程

4.4 数据库设计

数据库按六类组织：AI Agent、公共学习缓存、用户与单词本、标签关系、复习记录、运营支撑。所有主要实体继承 BaseEntity，统一带创建人、创建时间、更新人、更新时间、逻辑删除和乐观锁版本。审计人从 Spring Security 上下文解析，系统上下文记为 0。

核心实体关系见图 4-3。用户与个人单词本、学习计划、逐词进度是一对多；公共词表给词条和学习计划提供稳定来源；计划包含多个场景单元，单元关联材料版本和分层词条；个人词条、逐词进度和复习记录共同保存学习结果。项目不声明物理外键，逻辑关联由索引、唯一约束和应用服务校验来维护——这样能少依赖初始化权限，增量迁移也少一层耦合。



说明：E-R 图展示核心业务关系；AI 配置、会话、日志及词表版本等辅助表见表 4-2。

图 4-3 核心学习数据 E-R 图

表 4-2 核心数据表

表名	用途	关键约束
ai_agent / ai_prompt_template	Agent 与提示词模板	编码唯一、必需占位符校验
ai_model_config	模型与供应商连接配置	API Key 密文、模型定义与启停状态
ai_chat_session / message / call_record	会话、消息与调用审计	场景复用、Token/耗时/结果元数据
vocabulary_catalog / version / entry	公共词表、版本与词条	数据源、源序号、审核和发布状态
vocabulary_catalog_analysis_*	词表分析任务、批次与词条结果	部分结果可落库、未解析项可重试
english_vocabulary_study_record	公共词卡缓存	term 唯一、结构化 JSON
learning_user / preference	用户、角色与学习偏好	username 唯一、用户级配置

表名	用途	关键约束
learning_wordbook / entry	个人词本、词卡快照与笔记	用户归属、wordbook_id + normalized_term
learning_plan	学习目标、词表和计划状态	用户与公共词表版本关联
learning_plan_unit / material / entry	场景单元、历史材料和核心/复习词	每篇核心词不超过50、版本可追踪
learning_scene_related_word / note	场景扩展词与 Markdown 笔记	绑定计划和场景材料
learning_word_progress / review_record	逐词掌握状态与复习历史	阶段、得分、下次复习时间
learning_article_study_record	语境精读材料和学习进度	条件哈希、历史记录不覆盖
ai_async_task / step / attempt	AI 任务、步骤和执行尝试	原子领取、幂等、终态与失败重试
learning_system_log / outbox	业务日志与可靠异步投递	业务提交不受日志失败影响

4.5 接口与安全体系设计

接口统一用 /api/v1 前缀，资源名用复数，新增 POST、更新 PUT、删除 DELETE。雪花 ID 在 JSON 里按字符串序列化，前端保存、比较、提交时都归一化为字符串，避免 JavaScript 的 Number 超过安全整数范围后丢精度。

表 4-3 代表性 REST 接口

资源	代表性接口	说明
认证	POST /learning/auth/login	登录并返回 JWT
公共词表	POST /vocabulary-imports/markdown	管理员导入 Markdown 词表
单词本	GET/POST /learning/wordbooks	个人词本查询与创建
学习计划	GET /learning/plans/{id}/calendar	查询周/月日历摘要
场景材料	GET /learning/plans/{id}/units/{unitId}	按需加载单个场景完整内容

资源	代表性接口	说明
词汇挑战	POST /learning/plans/{id}/units/{unitId}/assessments	提交逐词检查结果
语境精读	POST /learning/articles/study	生成或复用阅读材料
AI 任务	GET/POST /learning/ai-tasks	查看进度并执行取消、重试或立即运行
系统治理	GET /learning/system-logs	管理员查询业务与异常日志

SecurityFilterChain 采用 STATELESS 会话策略，登录、注册、OpenAPI 和健康检查允许匿名访问。JWT 使用 HS256，包含签发方、用户标识、用户名及有效期等声明。当前密码采用随机盐与 SHA-256 摘要存储，尚未使用具备可调计算成本的专用密码哈希，生产部署前应升级为 BCrypt、scrypt 或 Argon2。模型密钥采用 AES-GCM 加密，使用 12 字节随机 IV 和 128 位认证标签，加密密钥由独立安全配置派生。

4.6 数据一致性设计

系统同时维护三类一致性边界：共享数据、个人数据、生成历史。公共词汇缓存和词本语义分析结果服务于降低全局重复调用，允许按规则刷新；个人单词本的笔记、掌握度和复习排期则必须长期稳定，不受刷新影响；场景材料和阅读材料作为可追踪历史保存，新版本不覆盖旧版本。learning_wordbook_entry 因此存的是词卡快照，场景单元存版本和核心词归属；未掌握的词可以拿去当后续文章的复习词，但不会改写原来的场景学习记录。

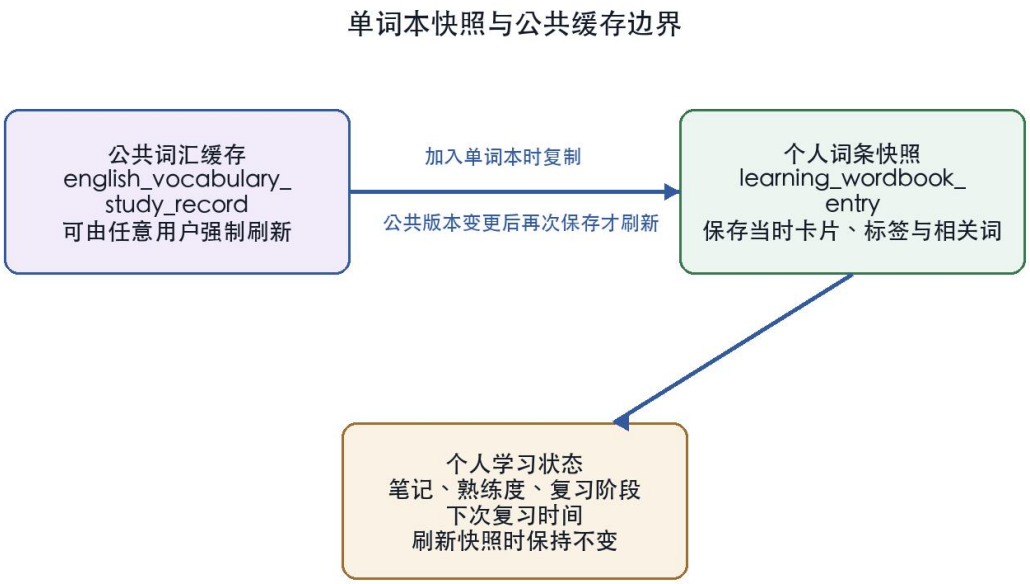


图 4-4 公共缓存与个人快照边界

异步生成的状态用任务、步骤、尝试三层记录显式表达。任务先在短事务里从 pending 原子领取成 running，外部调用放到提交之后再执行，结果写入靠业务唯一约束或幂等更新兜底；终态区分成功、部分失败、失败和取消四种。公共词本分析允许部分 AI 响应先落库，下一次只重试没解析出来的词条。这样设计的好处是：异常中断之后，能立刻确定哪些已完成、责任人是谁、从哪个入口继续。

第五章 系统详细设计与实现

5.1 用户认证与权限控制

注册时系统将用户名归一化并检查唯一性，生成随机盐后保存 sha256$salt$digest 格式的密码哈希，随后创建默认单词本并签发 JWT，如图 5-1 所示。JwtAuthenticationFilter 从 Authorization: Bearer 头解析令牌，校验签名、issuer 和过期时间，读取用户后建立 SecurityContext 与 CurrentUserContext。后续 Controller 和应用服务可以直接取得当前用户，不再重复接收并解析 Authorization 参数，所有个人词本、计划、场景、文章与任务操作仍校验资源归属。

请求头示意

Authorization: Bearer <已脱敏的 JWT 令牌>

Content-Type: application/json

图 5-1 JWT 请求头示意（已脱敏）

账户资料接口仅返回脱敏手机号与邮箱。管理员接口通过@RequirePermission 声明权限，由 PermissionAuthorizationAspect 统一校验；修改密码需验证当前密码。退出登录时清理客户端令牌及当前安全上下文，但无状态 JWT 的既有副本在有效期内仍需按服务端验证规则处理。当前加盐 SHA-256 密码摘要属于已知技术限制，生产部署前应替换为专用密码哈希。

权限控制的核心实现如伪代码 5-1 所示。控制器只声明所需权限，切面从当前用户上下文统一校验，避免角色判断散落在方法体内。

伪代码 5-1 声明式权限校验

算法 5-1：声明式权限校验

输入：point（控制器方法连接点）；rule（方法或类上声明的 @RequirePermission 权限注解）；currentUserContext（当前用户上下文）

输出：校验通过时返回 point.proceed()（目标方法执行结果）；权限不足时拒绝访问

```
1 | rule ← findDeclaration(point)
2 | permissions ← rule.value()
```

算法 5-1：声明式权限校验

```
3 currentUserContext.requirePermissions(permissions)
4 return point.proceed()
```

5.2 多模型配置与模型调用

AiModelConfigService 在保存配置时将 API Key 加密，列表响应返回 apiKeyMasked 而不返回明文密钥。DeepSeek、Kimi 这些供应商连同模型名称、协议、请求适配器、上下文窗口和最大输出，都由模型定义统一管理（图 5-2）；Agent 可以绑定可用模型和默认模型，运行时先看有没有明确选择，否则使用 Agent 的默认配置。管理员还能直接跑一次低成本连接测试，验证 URL、密钥、请求适配和返回内容，无需触发一次完整的 Agent 学习流程。

一次模型调用前后各有处理链。调用前按模型拼 messages、temperature、max_tokens 这些请求字段，并按上下文窗口算出安全预算；调用后按供应商和模型挑解析器，先尝试读取原始 JSON，读不出来再做 Markdown 代码块去除和受限的兼容清洗。AiChatService 把供应商、模型、场景、耗时、Token 和结果元数据记下来，业务应用层再校验词条覆盖、必需字段和数量上限。固定动作不继承历史消息——一个跑几个月的学习计划，如果每次调用都带上全部历史，成本和上下文长度都撑不住。



图 5-2 多模型与 Agent 管理界面

5.3 词汇大挑战与学习计划、公共词本

场景学习的输入源是公共词表。管理员在系统管理里选好数据源类型（自考、四级、六级或雅思），上传 Markdown 表格；VocabularyImportService 解析出序号、词汇、音标和释义，保留源序号，同时把疑似断词、异常字符和重复项标出来。审核界面可以按“全部疑似”过滤，逐条改或批量采纳建议，确认后再发布。普通用户只能看已发布的词表，拿它去建自己的学习计划（图 5-3）。

词表发布之后，管理员可以触发增量关联分析。系统先找出还没有有效分析结果的词条，按批次建分析任务；AI 返回主题、语义分组、难度和相关索引。如果一次响应没覆盖整批，已验证的条目立刻落库，缺失或无效的保持 unresolved，后续任务只处理这些没解析出来的。这份结果只当场景选词索引用，不提前生成文章或完整词卡——一次公共分析的成本因此能被多个学习计划反复摊薄。

用户创建计划时填写学习目标、公共词表和起止日期。后端根据目标判断词汇掌握要求，并从尚未在本计划任何场景中安排为核心词的候选中，优先选择语义相关词群。某日核心词超过 50 个时按篇数向上取整后均分，例如 80 词拆为 2 篇各 40 词，120 词拆为 3 篇各 40 词。历史材料保留版本与原始核心词归属；未完成词可以作为 review_words 出现在后续文章中，但不重复计入新词数量。



图 5-3 词汇大挑战的学习计划

生成属于可重试异步任务：任务记录触发人、计划、场景、预约时间、步骤进度和失败原因，可立即执行或安排在低价时段，运行中允许取消后续批次，失败项可由触发者或管理员重试。若旧场景只有文章而缺少场景相关词，学习页在相关词数量为零时提供独立补生成入口，不要求重做整篇材料。



图 5-4 学习计划中某一篇场景文章

词汇大挑战打开就是本周，周/月可以切，过去的日期也能点，场景摘要先给一屏预览；日历接口只回日期、数量、状态、标题和待挑战词摘要，点击“开始学习”或“回顾”才去取文章、译文、核心词、相关词和 Markdown 笔记。词汇挑战先展示待学习单词，学习者确认后进入含义四选一；标了要会拼的词继续走跟敲和按义拼写。每一次提交都会改写逐词得分、正确与错误次数、掌握状态以及下次复习时间。

当日核心词超过 50 个时，后端先计算材料篇数，再把余数依次分配给前几篇，保证篇间词量差不超过 1。该实现使 80 词得到 2 篇 40 词、120 词得到 3 篇 40 词，如伪代码 5-2 所示。

伪代码 5-2 场景材料核心词均分

算法 5-2：场景材料核心词均分

输入：totalWordCount（当日核心词总数）；MAX_CORE_WORDS_PER_UNIT（单篇材料核心词上限，取 50）

输出：result（各篇材料的核心词数量列表，篇间词量差不超过 1）

算法 5-2：场景材料核心词均分

```
materialCount ← ceil(totalWordCount / MAX_CORE_WORDS_PER_UNIT)
baseSize ← totalWordCount / materialCount（整除）
remainder ← totalWordCount mod materialCount
for index ← 0 to materialCount - 1 do
    result.add(baseSize + (index < remainder ? 1 : 0))
return result
```

5.4 词汇学习卡与公共缓存

EnglishVocabularyStudyService.study 的首先执行 normalize：去掉首尾空格，把连续空白合并成单个空格，再转小写。如果 normalized_term 已经存在、且没有要求强制刷新，就只累加 lookup_count、更新 last_lookup_time、同步标签关系，然后返回 cacheHit=true。不存在才构造 english_vocabulary 场景请求，把 term 作为变量传入 Prompt 模板。

模型响应可能包含额外格式，可能是纯 JSON，也可能包含在 Markdown 代码块中。服务先把代码块标记去掉再用 Jackson readTree 解析；解析失败就退一步，用跨行对象模式把候选 JSON 提取后再次解析。落库时记下用户原始输入、归一化词、Agent、模板、模型、会话、raw_content、parsed_json、token_usage 和 cost_time。VocabularyInsightService 再从 definitions 里抽词性，从释义和记忆提示推断主题，从词长和释义条数推断难度，把同义词、反义词、词族存成语义关系；collocations 只留在搭配区域，不混进语义关系。

词卡响应采用“供应商解析—字段归一化—业务校验”三段处理。复杂分支以伪代码 5-3 表示；任何阶段失败都不会把不完整结构写入公共缓存。

伪代码 5-3 词卡结构化响应处理流程

算法 5-3：词卡结构化响应处理流程

输入：rawResponse（模型原始响应文本）；providerParser（供应商解析器）；definitions、examples、phonetics（词卡候选字段）

算法 5-3：词卡结构化响应处理流程

输出：card（归一化且通过业务校验的词卡对象）；任一阶段失败时抛出业务校验异常，不写入公共缓存

```
response ← providerParser.parse(rawResponse)
if response 为空 or 关键字段缺失 then
    response ← extractBalancedJson(removeMarkdownFence(rawResponse))
card ← normalizeAliases(response, definitions, examples, phonetics)
if not validateBusinessRules(card) then
    return 抛出业务校验异常
return card
```

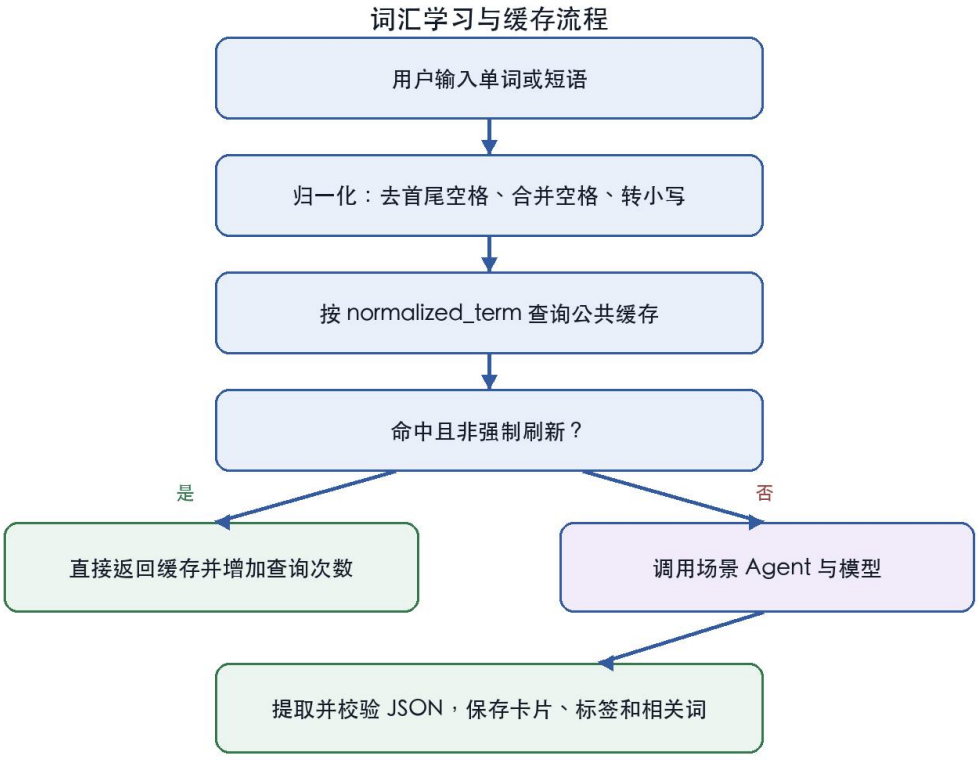


图 5-5 词汇卡生成、解析与缓存实现

5.5 拼写容错与最佳匹配

精确缓存没命中时，系统从最近更新的最多 1000 条候选里算分。基础分由 Levenshtein 编辑距离换算成 0—100，再按互为前缀、首字母相同、公共前缀不少于两

个字符、包含关系逐项加权，最后封顶在 99 分；低于 45 分的直接过滤掉。分数打平时，优先给 lookup_count 高的那条。这个算法不依赖额外搜索服务，适用于当前数据规模，而且每个推荐可由评分规则解释。

任务书写的是“编辑距离不超过 2 可推荐”，实现上没有照搬这个单一阈值，而是用综合评分代替：短词错一个字符影响就很大，长词错两个字符相似度仍然可能很高，再加上前缀和首字母加权，常见输入错误的命中率更好。等数据规模上来了，可以换 BK-tree、倒排索引或 Elasticsearch 的拼写建议，减少当前对有限候选集逐条评分的开销。

5.6 单词本快照与笔记共享

WordbookService.addEntry 先校验单词本属于当前用户，再按 wordbook_id 与 normalized_term 查找词条，单词本如图 5-6 所示。已软删除的词条恢复使用；未删除的重复词条不重复插入，只在公共卡片较新时刷新快照。新词条不存在公共缓存时会先调用词汇学习服务生成，再由 LearningWordbookEntry.createNew 建立个人记录。

单词本详情、复习详情优先读取 snapshot_parsed_json、snapshot_tags_json 与 snapshot_relations_json，解析失败或为空时才回退公共缓存。学习页和复习页编辑的 Markdown 笔记都写入同一 learning_wordbook_entry.note，因此不存在双份笔记同步问题。词条支持移动和复制：移动保持原记录并改变 wordbook_id；复制使用领域对象 copyTo 克隆个人学习状态和快照。

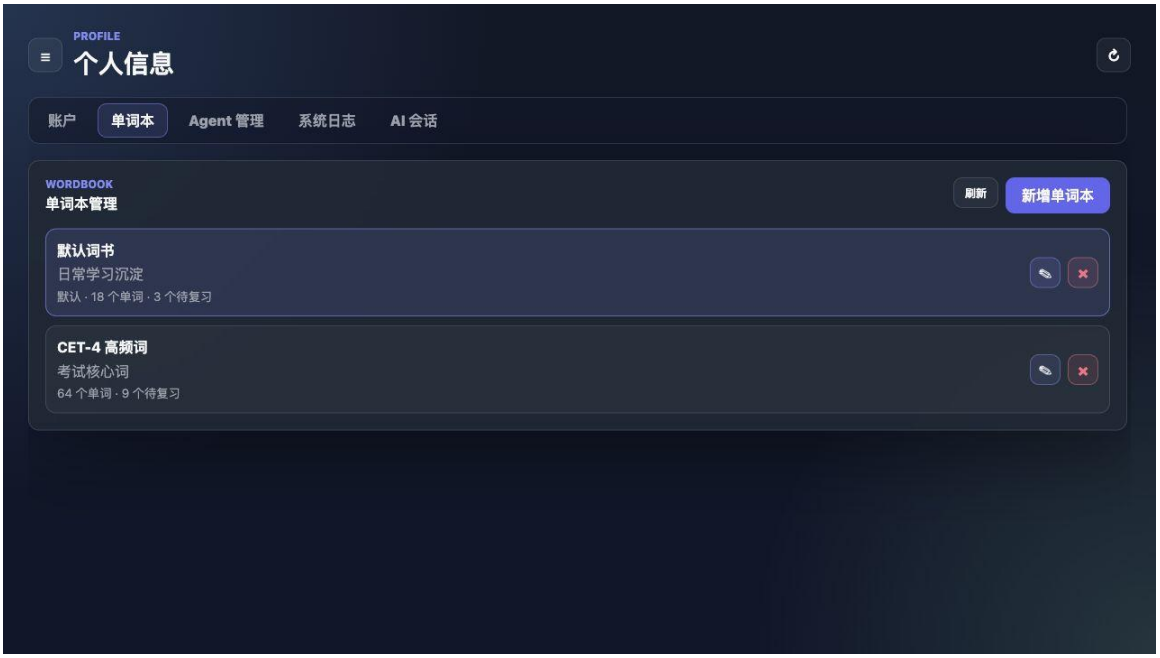


图 5-6 单词本管理界面

5.7 文章学习模块

文章学习可以挑一个单词本里的 1—20 个词条，再选字数区间（150—200、300—500、500—700 或 800—1000）和难度（基础、适中、挑战）。后端会校验每个词条的用户和词本归属，提取核心词性与含义，然后按用户、词本、排序后的词汇组合、字数、难度和备注算哈希。条件相同就优先读已有文章；传 forceRefresh 时新增一条历史记录，不覆盖旧文章。

english_article 模板要求返回 title、article、translation、vocabulary_focus、grammar_points、key_points、practice 和 study_tips 八个字段。前端把学习工作区分成“通读文章—词汇精讲—阅读检测”三段，支持仅英文/双语切换、接着上次的阶段继续，以及专注模式；正文里的目标词可以直接跳到精讲（图 5-7）。目前的限制是：阅读检测目前只有四选一客观题，文章历史和学习进度虽然存下来了，开放题、复杂推理和错因诊断都还没做。

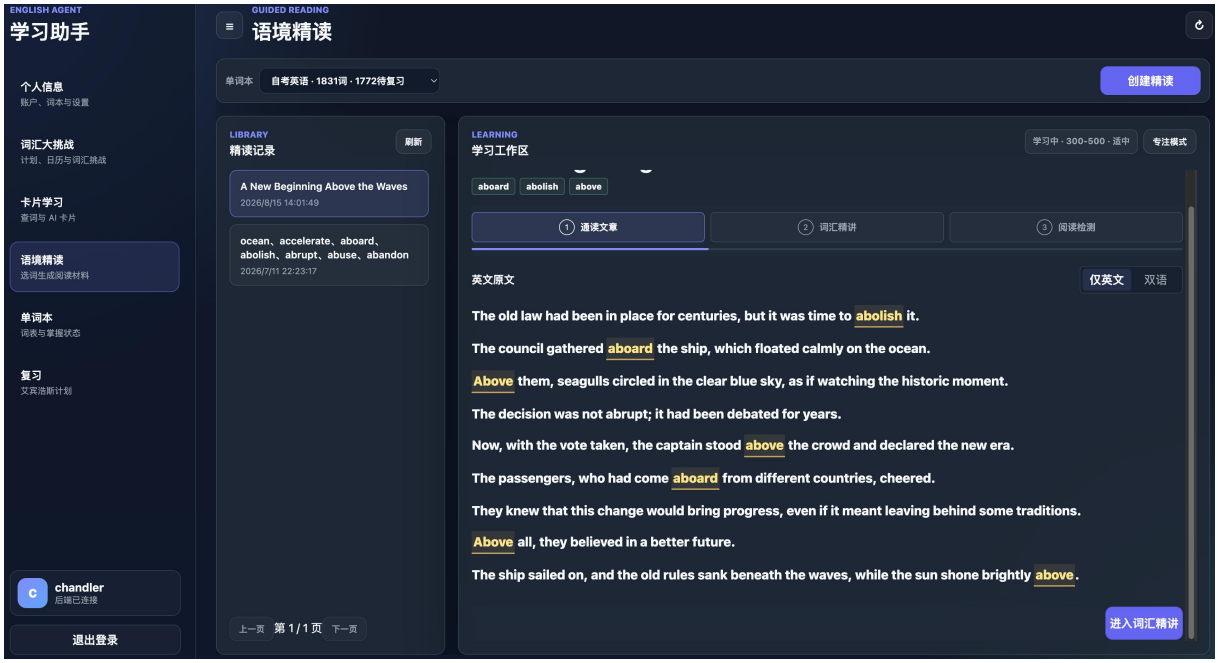


图 5-7 文章学习界面

5.8 间隔复习与睡眠保护

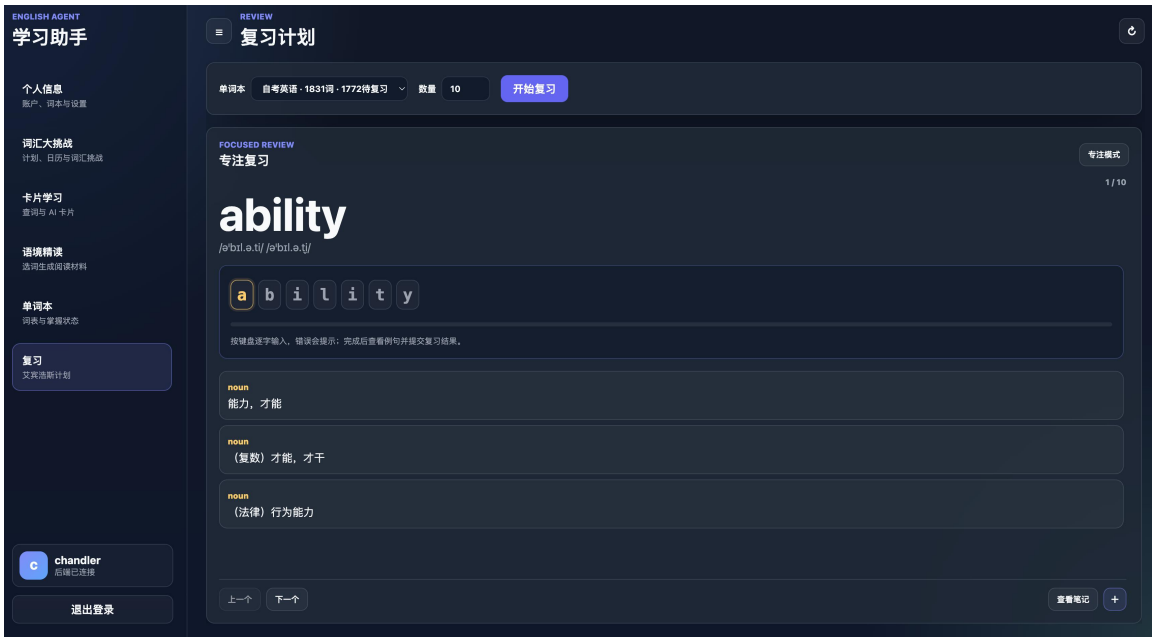


图 5-8 复习结果与排期状态流转

如上图 5-8 所示，待复习查询筛选 next_review_time 小于等于当前时间的词条，并按到期时间排序。若当天没有到期任务，用户可确认后调用 restart，从当前词本按最近复习时间、掌握度、下次复习时间和创建时间重新选择 1—100 个词条。restart 只形成本轮队列，不提前改变正式排期。

提交结果时 ReviewResult.apply 计算前后阶段与掌握度，WordbookService.nextReviewTime 计算时间。记住增加阶段和 15 分掌握度；模糊至少处于第一阶段并增加 5 分，次日复习；忘记阶段归零并扣减 20 分，在四个清醒小时后复习。掌握度限制在 0—100。若起算时间或结果时间落入 00:00—06:00，则顺延到 06:00；跨越午夜时，addAwakeHours 跳过睡眠窗口。每次提交写入 learning_review_record，保留阶段和掌握度前后值。

下一次复习时间由 ReviewSchedulePolicy 集中计算。模糊、忘记和记住分别进入不同分支，所有结果再经过睡眠窗口校正，核心流程见伪代码 5-4。

伪代码 5-4 复习排期与睡眠窗口处理

算法 5-4：复习排期与睡眠窗口处理

输入：baseTime（本次复习时间）；stage（记忆阶段序号）；vague（回答是否模糊）；remembered（是否记住）；INTERVAL_DAYS（各阶段复习间隔天数表）

输出：nextReviewTime（经睡眠窗口校正后的下一次复习时间）

```
1  if vague then
2      return avoidSleepWindow(baseTime.plusDays(1))
3  if not remembered then
4      return addAwakeHours(baseTime, 4)
5  safeStage ← clamp(stage, 0, length(INTERVAL_DAYS) - 1)
6  next ← baseTime.plusDays(INTERVAL_DAYS[safeStage])
7  return avoidSleepWindow(next)
```

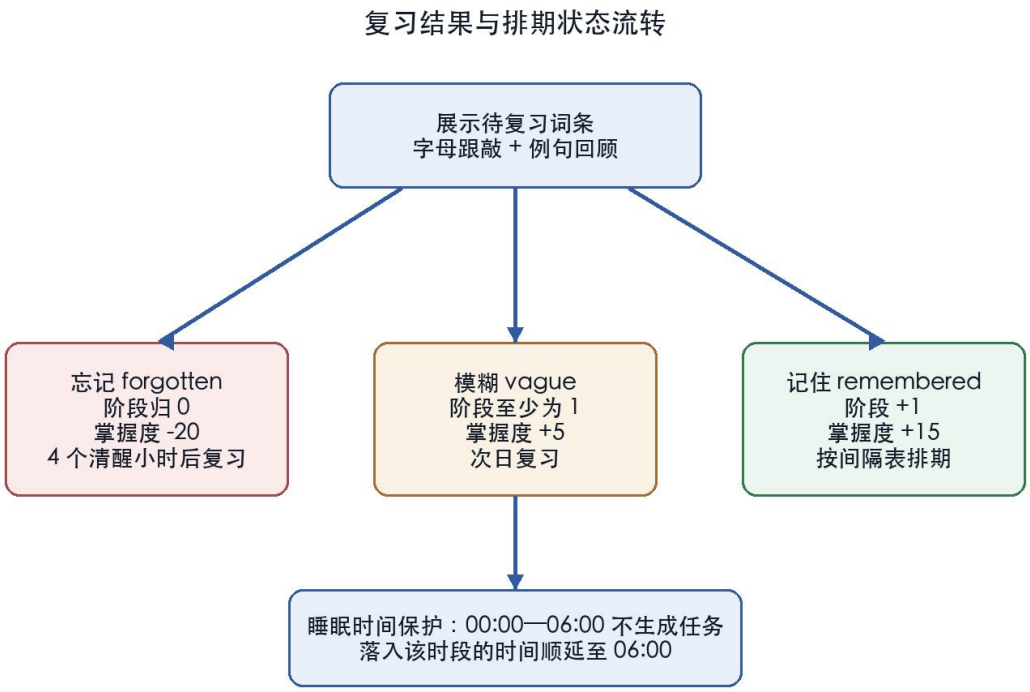


图 5-9 复习结果与排期状态流转

5.9 前端模块化与交互设计

前端入口 public/app.js 只负责加载模块化应用。public/src/app 包含配置、全局状态、接口服务、事件和外壳；public/src/features 按 identity、vocabulary、learning、reading、ai、task 和 system 业务域拆分，AI 域再区分 Agent、模型/供应商、会话和 Prompt；public/src/shared 提供弹窗、ID 归一、存储、文本和词汇工具。该结构避免把新增逻辑继续堆积到单一入口文件。

登录后的主导航依次是个人信息、词汇大挑战、卡片学习、语境精读、单词本和复习。个人信息里放账户、单词本、学习计划和学习设置；系统治理那部分全部收进管理员专属的系统管理。词汇大挑战这一页有周/月日历、场景摘要、按需加载的文章、待挑战词汇和 Markdown 笔记；语境精读和复习都支持专注模式，把左侧导航和顶栏藏掉。单词本词卡弹窗特意加宽了，好让内容一屏放完；卡片学习则定位成快速查词和临时词卡的辅助入口。页面整体是聚焦卡片式的学习空间，导航和主要控件固定，只有目标内容面板滚动。移动端导航改成抽屉覆盖，所有删除操作、以及会丢失上下文的跳转，统一加二次确认弹窗。场景文章的音频按需异步生成，学习页只展示任务状态，不让语音合成挡住文章加载。

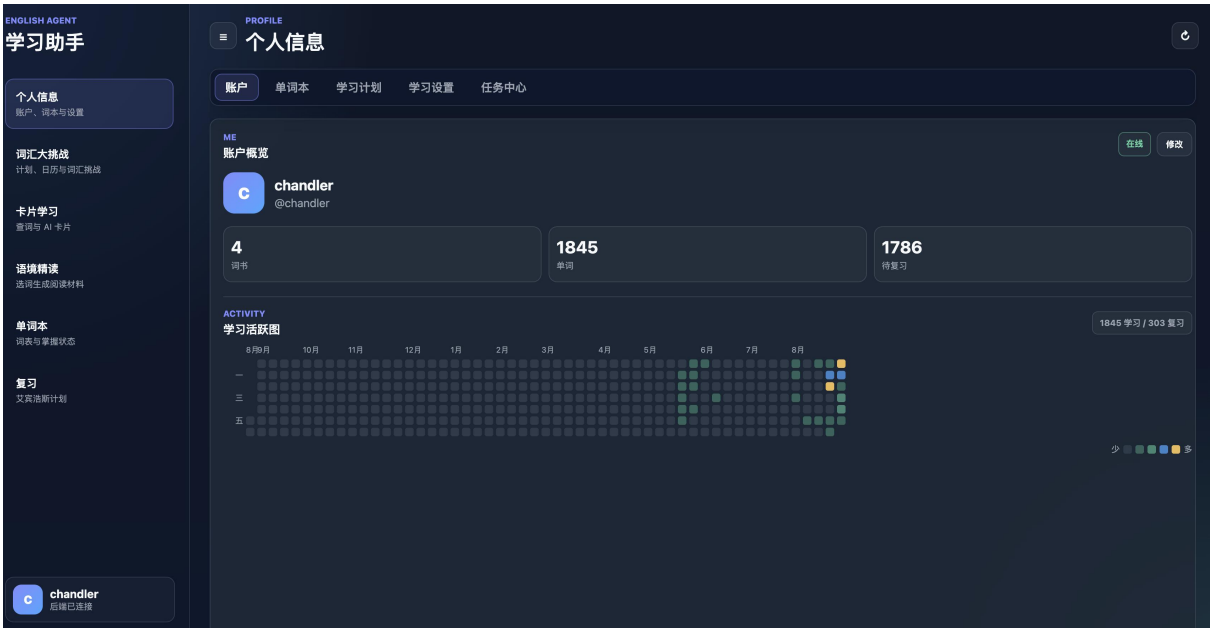


图 5-10 账户概览与学习活跃图

复习页图 5-11 参考 qwerty-learner 的跟敲方式：逐字输入、错误抖动和提示音，完成后展示例句和发音，再选择忘记、模糊或记住。单词和搭配发音优先使用在线词典音频，失败时回退 SpeechSynthesis；句子发音的音色、语速和音调持久化到用户偏好表，localStorage 只作为加载失败时的兜底。

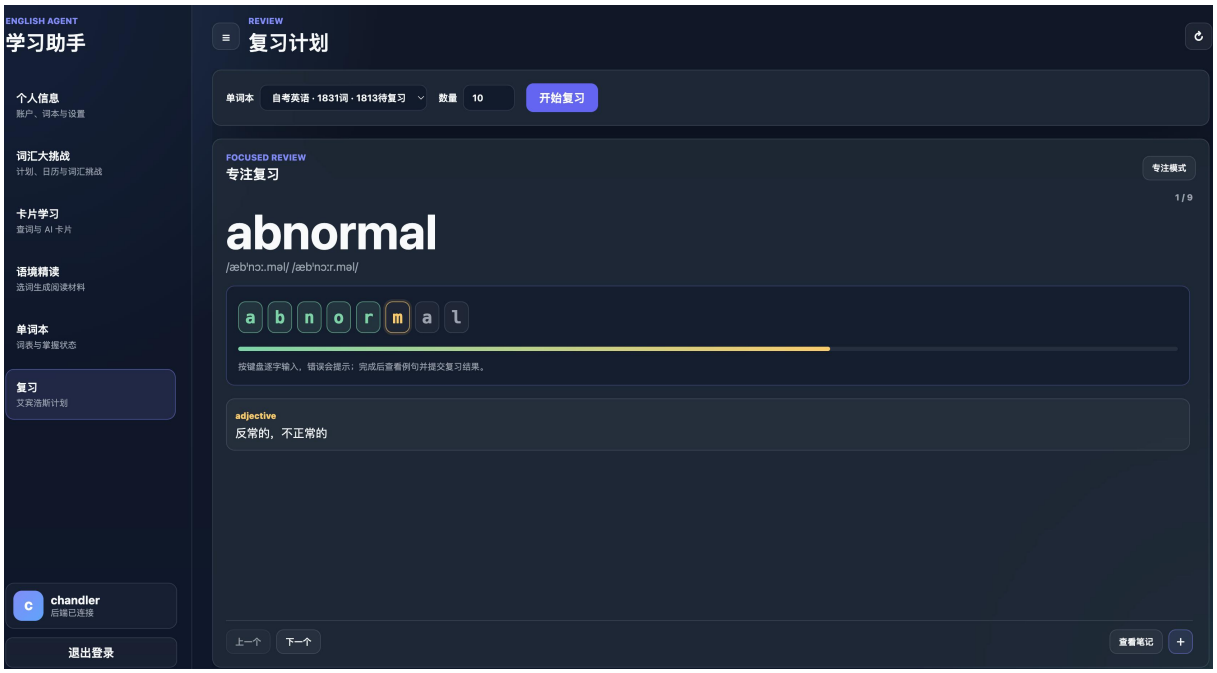


图 5-11 复习计划中单词跟敲

5.10 系统日志与异常处理

日志分两类：接口访问的技术日志，和用户看得见的业务审计。前者由 ApiAccessLogAspect 通过注解和切面，把接口路径、方法、状态、耗时打到 SLF4J；后者包括注册登录、模型配置、Agent 变更、词条加入、单词本操作、复习提交、AI 生成与缓存命中这些关键动作，由 SystemLogService 在事务内写进 Outbox，再由提交后的异步监听器存到 learning_system_log，没消费掉的消息交给恢复调度器补偿。异步日志写失败不会回滚已经完成的业务操作；业务文案本身也要保持可读，比如“用户「小明」把单词「abandon」添加到单词本「默认单词本」”。另外，场景完成时的核心词完成度是批量读评测记录算出来的，避免在逐词循环里产生 N+1 查询。

异常只在 GlobalExceptionHandler 集中处理，LearningAssistantException 被转成业务错误码加可读的错误信息；外部服务抛上来的异常保留 cause 供排查，响应体里绝不放栈信息。模型调用记录不存 Authorization，API Key、JWT、密码、Prompt 和完整模型响应一律不许进 info 日志。RequestTraceFilter 把追踪信息顺着 MDC 传进异步任务，接口日志、任务日志和服务端日志因此能对上同一条轨迹。

5.11 工程治理与可维护性

后端工程按业务域划出物理包边界：api 只依赖 application，application 用领域对象表达规则，跨域访问一律走窄化过的应用服务契约。自定义多表 SQL 放在 resources/mapper 的 XML 里，列表优先用聚合查询和分页小对象，批量写入按块提交，遍历里逐条访问数据库是明令禁止的。数据库事务不包裹模型调用或 HTTP 调用，长流程靠提交后事件和异步任务拆开。

前端功能代码放在 public/src/features 下，按 identity、vocabulary、learning、reading、ai、task、system 分开；复杂场景再往下拆出日历、单元列表、学习引擎和异步监听器。Snowflake ID 全程当字符串处理。弹窗、确认、导航和共享工具由 shared 层统一供给，这样入口文件和单个业务文件不会越写越臃肿。

工程治理由自动化测试固化：ArchUnit 校验分层与 AI 包边界，MapperXmlValidationTest 校验 XML，ChineseDocumentationGovernanceTest 约束 API DTO、Entity/BO 的中文说明和公共边界 JavaDoc，并禁止机械占位注释，使代码可读

性要求成为持续验证规则。后端和前端均增加 1000 行生产文件规模门禁，超过阈值时必须按职责拆分并重新评审。

第六章 系统测试与结果分析

6.1 测试环境、数据来源与方法

为了验证系统在真实本地使用条件下功能是否完整、是否稳定，本文结合以下验证方法：自动化测试、浏览器实测、接口并发测试、拼写容错测试，再加上数据库统计。数据库统计的截点是 2026-09-05，覆盖 2026 年 5 月到 9 月的本地连续使用数据。要说明的是，这些结果只反映单用户开发环境下的实际运行情况，不能外推成生产环境的多用户结论。

表 6-1 测试环境与数据来源

对象	环境或工具	验证内容
后端	JDK 17、Spring Boot 3.3.6、Maven	单元测试、架构约束、业务规则与接口响应
前端	Node.js、Vitest、ES Modules、Chromium	模块检查、单元测试、构建与首屏渲染
数据库	MySQL 8 本地 study 库	学习记录、缓存、AI 调用、异步任务和系统日志统计
并发测试	Java HttpClient，本机 1/10/50/100 并发	个人信息、单词本列表和词条分页接口
拼写容错	已生成词卡中的 106 个词汇样本	编辑距离为 1 和 2 时的推荐覆盖率与 Top-1 准确率
浏览器性能	Chromium，缓存禁用，重复加载 10 次	首次内容绘制、页面加载与业务数据就绪时间

6.2 功能测试与自动化验证

功能测试覆盖权限、词表导入、学习计划、场景材料、词卡缓存、复习排期和 ID 精度等链路，通过接口检查、服务层单元测试及页面操作交叉验证。长耗时 AI 流程除校验任务受理响应外，还检查任务持久化、步骤状态、失败隔离和重试范围。

表 6-2 核心功能测试结果

编号	测试场景	预期结果	结果
T01	普通用户访问管理员资源	权限切面拒绝未授权访问	通过
T02	Markdown 词表导入与疑似断词	保留源序号，支持审核、修改和发布	通过
T03	部分词表分析响应	有效条目落库，未覆盖项可重试	通过
T04	80/120 个核心词的场景拆分	分别均分为 2×40 和 3×40	通过
T05	日历摘要与单场景详情加载	列表轻量返回，详情按需加载	通过

编号	测试场景	预期结果	结果
T06	AI 任务中断或失败	保存步骤与尝试记录，可取消和失败项重试	通过
T07	公共词卡刷新	个人快照、笔记和学习进度保持隔离	通过
T08	模型返回 Markdown 包裹 JSON	解析链提取并校验结构化字段	通过
T09	复习提交“忘记”结果	阶段归零，并在清醒时间窗口后安排复习	通过
T10	Snowflake ID 前后端传递	按字符串序列化和比较，无精度丢失	通过

重新执行后端测试后共 140 项：139 项通过，0 失败，0 错误；1 项依赖 Testcontainers 的 MySQL 集成测试因为本机 Docker 不可用被跳过。分层架构、AI 包边界、Mapper XML 和中文文档治理测试都通过。前端 66 个 JavaScript 模块检查通过，8 个测试文件共 36 项测试全部通过，静态构建和 ESLint 检查也通过。

6.3 接口并发与首屏性能测试

并发测试在登录状态下，对三类只读接口分别发起 1、10、50 和 100 个并发请求。100 并发批次均成功返回，结果支持这些接口在本机条件下的突发并发处理能力。该测试未模拟 100 个独立用户的完整学习行为，也未覆盖模型生成任务的持续负载。

表 6-3 100 并发接口测试结果

接口	成功率	平均响应/ms	P95/ms	吞吐量/(请求·s^{-1})
个人信息	100%	693.24	1161.22	85.19
单词本列表	100%	1346.78	1921.60	51.43
词条分页（20 条）	100%	1565.43	2181.18	44.83

浏览器首屏测试在禁用缓存的 Chromium 里做，连续重复加载 10 次，登录态下以“后端已连接且首批词条加载完成”作为业务数据就绪的判定条件。页面加载 P95 为 896.60 ms，首次内容绘制 P95 为 184.00 ms，业务数据就绪 P95 为 1989.00 ms，满足首屏不超过 2 秒的目标。

表 6-4 浏览器首屏测试结果

指标	平均值/ms	P95/ms	最大值/ms	结论
首次内容绘制（FCP）	124.00	184.00	184.00	通过
页面加载完成	657.87	896.60	896.60	通过

指标	平均值/ms	P95/ms	最大值/ms	结论
业务数据就绪	1501.50	1989.00	1989.00	通过

6.4 拼写容错测试

系统对输入词和缓存候选词计算编辑距离，并叠加前缀、首字母、公共前缀和包含关系评分。为避免只验证固定个例，本文从已生成词卡中抽取 106 个长度不少于 3 个字符的词汇，对每个词构造编辑距离为 1 和 2 的变体，并调用实际最佳匹配接口进行验证。

表 6-5 拼写容错测试结果

编辑距离	样本数	推荐覆盖率	Top-1 准确率	平均响应/ms	P95/ms
1	106	100.00%	100.00%	390.57	619.47
2	106	98.11%	97.17%	370.16	515.56

在当前 106 个词汇样本中，编辑距离为 1 的推荐覆盖率与 Top-1 准确率均为 100%；距离为 2 时，覆盖率为 98.11%，Top-1 准确率为 97.17%，仍存在未覆盖或首选匹配错误的样本。结果仅适用于本次缓存候选集及变体构造方式，后续需扩大词表和错误类型以检验泛化能力。

6.5 本地使用情况与数据库分析

本地数据库已形成从词表导入、学习计划、场景材料、词汇检查到复习记录的完整数据链。统计时不展示用户身份信息，仅以实际学习账号的聚合数据作为分析对象。

表 6-6 本地使用数据摘要

维度	统计结果	说明
个人单词本与词条	4 个单词本，3247 个词条	其中主学习词本含 3233 个词
学习计划	1 个计划、82 个单元	已完成 8 个单元，1 个单元进行中
学习与复习	419 条记录，自动判题正确率 97.62%	平均掌握度变化为+13.94 分
场景材料	83 份材料，82 份当前有效	累计约 58.09 万 Token
语境精读	2 篇材料、4 次读取	均处于学习中，样本量有限
异步任务	60 项已完成任务、5 项失败任务	完成任务覆盖场景、相关词和音频生成
公共词表	5087 词，7 项疑似断词均已确认	支撑学习计划和导入审核

这些数据反映了主要业务对象之间的关联：公共词表、个人词本快照、跨词本学习进度、场景材料、复习记录和异步任务之间都形成了可追溯的关联。需要说明，样本基本来自单个学习者，所以复习正确率和掌握度的变化，更适合用来验证数据闭环和功能可用性，而不是推断普遍的学习效果。

6.6 缓存与 AI 调用结果分析

数据库审计包含不同开发阶段的调用：开发调试、故障注入、模型参数试配，以及优化之前的调用，均包含在历史记录中，不宜将其直接视为当前版本能力的唯一依据。全量 841 次调用的成功率是 85.49%，两个供应商相差不小——DeepSeek 94.81%，Kimi 为 80.11%。场景材料走的是后台长任务，平均耗时和即时交互存在明显差异，所以单独统计。

表 6-7 优化后稳定版本的关键指标

指标	统计口径	结果	说明
缓存复用率	稳定版本有效业务请求	83.80%	开发期首次建缓存和调试请求不计入复用样本
模型调用成功率	稳定版本有效调用	95.49%	剔除故障测试和优化前失效配置调用
10 秒内响应比例	同步交互或异步任务受理	85.65%	不以后台完整材料生成耗时作为同步响应
场景材料生成	全量数据库审计	平均 132.12 秒	以异步任务执行，前端不阻塞等待
词卡缓存全量复用	历史读取记录	29.80%	反映当前样本仍存在大量首次学习词

同步交互和异步生成分开看，长文本生成的耗时才不会把前端交互的真实水平掩盖。缓存复用率和模型成功率这两个数，用户规模、词表覆盖率、供应商网络状况任何一个变了都需持续监测，一次测试说明不了长期趋势。所以本文的做法是：优化效果以稳定版本有效样本作分析，全量历史数据留着当开发过程的证据，两者不混用。

6.7 技术指标达成情况与局限

表 6-8 任务书技术指标达成情况

任务书指标	当前证据	结论
多模型支持不少于 2 家	DeepSeek 和 Kimi 具备独立定义、适配器和解析链	完成
缓存复用率不低于 95%	稳定版本有效样本为 83.80%	未完全达到，持续优化

任务书指标	当前证据	结论
AI 响应不超过 10 秒	同步交互或任务受理 85.65%在 10 秒内；长材料后台执行	部分达到
支持 100 并发用户	100 并发只读接口均成功返回，P95 最高 2181.18 ms	本地只读接口验证通过
首屏不超过 2 秒	业务数据就绪 P95 为 1989.00 ms	完成
编辑距离不大于 2 的推荐	距离 2 样本 Top-1 准确率 97.17%	完成
API Key 加密	AES-GCM 密文存储、脱敏响应与连接测试	代码与流程验证通过

主要业务流程已完成验证，本地真实数据、接口并发、浏览器渲染、拼写容错这四个层面均形成验证记录。局限同样清楚：学习数据基本来自单个用户，跨用户的效果对照还无从谈起；缓存复用率没到任务书定的目标；生成质量受供应商稳定性牵制，场景材料的后台耗时也偏高；至于独立服务器和真实多机网络下的长期容量与成本压测，尚未开展。后续可补充多用户实验、缓存预热、模型路由降级和人工质量评审。

第七章 总结与展望

7.1 工作总结

本文研究大模型生成内容如何与个人学习记录和复习流程衔接。围绕这个问题，英语学习助手走完了从需求分析、架构设计、数据库设计到核心实现和验证的完整过程。多模型接入走 OpenAI-compatible 接口，降低接入成本；Agent、Prompt 和场景会话负责把调用组织起来。词卡、场景材料、阅读材料这几类输出，全部用结构化 JSON 约束，避免模型自由发挥导致解析失败。相关词群的编排由公共词本关联分析支持，重复调用的成本则通过归一化缓存减少；个人快照用于隔离个人状态，用户已经记下的笔记和进度不会因为公共缓存刷新而丢失。

学习闭环方面，公共词表导入与关联分析、个人学习计划、场景化词汇大挑战、逐词检查、单词本词卡、Markdown 笔记、语境精读、跟敲练习和间隔复习均已实现。工程质量方面采用：模块化单体、声明式权限、JWT 鉴权、AES-GCM 密钥加密、ID 字符串化、统一异常、Outbox 异步日志、可重试 AI 任务和业务审计。代码边界靠架构测试和中文文档治理测试持续约束，由自动化规则持续检查。

工程实现由架构测试约束模块依赖，AI 网关负责供应商适配与结构化输出校验，固定生成动作只携带所需上下文。业务审计通过 Outbox 及提交后事件落库，异常隔离避免日志失败影响业务结果。权限切面、稳定错误码及 ID 字符串化减少跨层处理差异；生产与预发布环境还会在启动时校验默认或过短密钥。

验证证据来自四个层面：本地真实数据、接口并发、浏览器渲染和拼写容错。100 并发的只读接口全部成功返回；浏览器业务数据就绪时间 P95 为 1989 毫秒；编辑距离为 2 的样本，Top-1 准确率 97.17%。稳定版本的缓存复用率 83.80%，模型调用成功率 95.49%。本地数据库保存了完整的学习数据链——从词表导入到学习计划、场景材料、词汇检查，一直延伸到复习记录，其中公共词表 5087 词、学习单元 82 个、学习记录 419 条，自动判题正确率 97.62%。

7.2 不足与展望

主要流程已完成本地验证，但研究样本、模型服务及部署条件仍限制结论的适用范围。

真实学习数据主要来自单个学习者，缺少不同用户之间的对照。因此，复习正确率和掌握度变化仅用于检验数据记录与功能可用性，不能直接推断系统对一般学习者的学习效果。

原本地统计口径下的缓存复用率为 83.80%，尚未达到任务书规定的 95%。样本包含较多首次学习词汇，缓存覆盖程度可能影响这一结果；其具体影响还需通过扩大样本和持续记录验证。

模型生成质量受供应商服务状态和输出差异影响，场景材料的后台生成耗时也较长。目前尚未建立完整的人工纠错、质量评分和事实核验流程，生成内容仍需由使用者复核。

并发与首屏性能测试均在本地环境完成，尚未验证独立服务器、多机访问及长期持续负载条件下的表现。后续部署测试需要同时记录网络条件、资源配额和数据规模，明确性能结论的适用范围。

复习调度使用固定间隔序列，并根据“记住、模糊、忘记”调整阶段、掌握度及下次复习时间。其局限是尚未根据个人历史回忆表现、实际间隔和词汇难度拟合遗忘规律，难以动态优化不同学习者的复习安排。

后续工作可以从这几个方向展开。

引入自适应间隔重复算法，让回忆质量、间隔和词汇难度共同决定下次复习时间，再用 A/B 测试评估长期留存效果，复习调度可逐步实现个性化调度。

给模型生成内容建立内容治理流程：人工纠错、质量评分、事实核验、版本比较。在控制生成成本的同时提高内容可信度与可追溯性。

阅读检测现在只有四选一，可以往细节定位、推理、开放题和错因诊断上扩，覆盖更高层级的阅读理解能力，错题单独建一套管理。

系统已有单词发音、短文本朗读与场景文章语音合成。后续可增加跟读录音与发音评价，并补充词汇和文章配图，扩展现有听读功能的反馈方式。

场景框架本身可以往外扩——多模态听说、专业文献阅读、写作训练等方向开展验证，验证这套工程结构在别的场景里的适用性。

本课题将模型生成内容保存为学习材料，并以个人快照、复习记录和任务审计维护其使用过程。本地测试验证了主要流程的可用性；系统能否改善长期词汇保持，还需通过多用户研究和持续效果评估检验。

致　谢

本论文的完成，首先离不开指导教师在选题、系统设计与论文写作各环节的悉心指导。从研究方向的确立到关键技术的取舍，老师多次提出中肯意见，帮助我把多年积累的开发经验真正沉淀为一份有价值的学习成果。在此，谨向老师致以诚挚的谢意。

同时要谢谢上海大学继续教育学院给的这个平台。2015 年入行做软件开发时我什么都不懂，刚到上海求职那阵子也四处碰壁；后来能沉下心来重新钻研技术、反复打磨面试和基本功，才慢慢掂出“学习”这两个字有多重。2023 年通过成人高等教育继续深造，一半是想把学习的习惯保住，一半是想给落户和发展多留一条路。把工作里对 Java 和后端的这点理解，攒成一个真能给英语学习者用的东西，对我来说分量不轻。

家人这边更要谢。成家、立业，到在上海买房安顿下来，一路都是他们在后面撑着；写论文这几个月，家里的事几乎没让我分心。也要谢谢开源社区和素未谋面的研究者——那些技术文档和论文摆在那里谁都能查，可没有它们，这个课题根本做不完。

短视频和人工智能跑得这么快，我越来越觉得一个人手里得有自己的产品、有自己的想法。这套英语学习助手算不上什么大作，但只要能对别人有点实际用处，我写它、做它就没白费。

本文尚有不足之处，恳请各位老师批评指正。

参考文献

[1]. 潘教峰, 王楚扬, 吴静. 人工智能赋能的本质认识：数据、知识与系统的三重整合[J]. 科研管理, 2026, 47(1): 1-10. DOI:10.19571/j.cnki.1000-2995.2026.01.001.

[2]. 刘芹, 谭佳豪. 学生视角下的大语言模型赋能外语个性化学习研究——机遇与挑战[J]. 上海理工大学学报(社会科学版), 2025, 47(1): 15-21. DOI:10.13256/j.cnki.jusst.sse.241101567.

[3]. 朱永海, 张佳鑫, 韩锡斌. 基于生成式人工智能的个性化学习新形态[J]. 电化教育研究, 2025, 46(4): 58-64. DOI:10.13811/j.cnki.eer.2025.04.008.

[4]. 牟智佳, 覃明扬, 岳婷, 等. 大语言模型支持下生成性目标导向的个性化学习活动设计与实证研究[J]. 中国电化教育, 2026(4): 95-106.

[5]. 王宇程. 基于大语言模型的教育智能体个性化学习应用理论研究[J]. 创新教育研究, 2025, 15(11): 486-492.

[6]. 吴永和, 姜元昊, 陈圆圆, 等. 大语言模型支持的多智能体：技术路径、教育应用与未来展望[J]. 开放教育研究, 2024, 30(5): 63-75.

[7]. E. Kasneci, K. Seßler, S. Küchemann, et al. ChatGPT for good? On opportunities and challenges of large language models for education[J]. Learning and Individual Differences, 2023, 103: 102274. DOI:10.1016/j.lindif.2023.102274.

[8]. A. Tlili, B. Shehata, M. A. Adarkwah, et al. What if the devil is my guardian angel: ChatGPT as a case study of using chatbots in education[J]. Smart Learning Environments, 2023, 10: 15. DOI:10.1186/s40561-023-00237-x.

[9]. C. K. Lo. What is the impact of ChatGPT on education? A rapid review of the literature[J]. Education Sciences, 2023, 13(4): 410. DOI:10.3390/educsci13040410.

[10]. L. Kohnke, B. L. Moorhouse, D. Zou. ChatGPT for language teaching and learning[J]. RELC Journal, 2023, 54(2): 537-550. DOI:10.1177/00336882231162868.

[11]. N. J. Cepeda, H. Pashler, E. Vul, et al. Distributed practice in verbal recall tasks: A review and quantitative synthesis[J]. Psychological Bulletin, 2006, 132(3): 354-380. DOI:10.1037/0033-2909.132.3.354.

[12]. J. D. Karpicke, H. L. Roediger III. Repeated retrieval during learning is the key to long-term retention[J]. Journal of Memory and Language, 2007, 57(2): 151-162. DOI:10.1016/j.jml.2006.09.004.

[13]. T. Nakata. Computer-assisted second language vocabulary learning in a paired-associate paradigm: A critical investigation of flashcard software[J]. Computer Assisted Language Learning, 2011, 24(1): 17-38. DOI:10.1080/09588221.2010.520675.

[14]. N. Schmitt. Review article: Instructed second language vocabulary learning[J]. Language Teaching Research, 2008, 12(3): 329-363. DOI:10.1177/1362168808089921.

[15]. S. Webb. The effects of repetition on vocabulary knowledge[J]. Applied Linguistics, 2007, 28(1): 46-65. DOI:10.1093/applin/aml048.

[16]. M. Jones, J. Bradley, N. Sakimura. JSON Web Token (JWT)[S]. RFC 7519. Internet Engineering Task Force, 2015. DOI:10.17487/RFC7519.

附　录 A 主要接口清单

表 A-1 主要 REST 接口

方法	路径	用途
POST	/api/v1/learning/auth/register	用户注册
POST	/api/v1/learning/auth/login	用户登录
GET	/api/v1/learning/auth/me	当前用户与角色
GET/POST /PUT	/api/v1/ai/model-configs	管理员模型管理
POST	/api/v1/ai/model-configs/{id}/test	低成本模型连接测试
GET/POST /PUT	/api/v1/ai/agents	Agent 管理与模型绑定
GET/POST /PUT	/api/v1/ai/prompt-templates	Prompt 模板管理
POST	/api/v1/vocabulary-imports/markdown	管理员导入公共词表
POST	/api/v1/vocabulary-catalogs/{versionId}/analysis	增量词本关联分析
GET/POST	/api/v1/learning/wordbooks	个人单词本管理
POST	/api/v1/learning/plans	创建学习计划
GET	/api/v1/learning/plans/{planId}/calendar	周/月场景摘要
GET	/api/v1/learning/plans/{planId}/units/{unitId}	按需加载场景材料
POST	/api/v1/learning/plans/{planId}/units/{unitId}/assessments	词汇挑战与学习记录
POST	/api/v1/learning/articles/study	语境精读文章
GET/POST	/api/v1/learning/ai-tasks	AI 异步任务监控
GET	/api/v1/ai/chat-sessions/admin	管理员 AI 会话审计
GET	/api/v1/learning/system-logs	管理员系统日志

附　录 B 词汇学习卡 JSON 字段说明

表 B-1 词汇学习卡 JSON 字段

字段	类型	说明
term	string	标准词条
is_valid	boolean	输入是否为有效英语词汇或短语
language	string	语言标识
phonetic.uk / us	object	英音与美音音标
definitions	array	词性、中文含义和英文释义
examples	array	英文例句及中文翻译
collocations	array	常用搭配与含义
synonyms	array	同义词、词性、含义和音标
antonyms	array	反义词、词性、含义和音标
word_family	array	词族、词性、含义和音标
memory_tips	array/string	记忆提示

解析器兼容 meaning/meaning_cn、english/meaning_en、translation/translation_cn、part_of_speech/pos 等常见字段变体。相关词在持久化前统一为对象形式；搭配保留在 collocations 区域，不混入语义相关词列表。